When Can the Planner Trust a Filtered Unique Index?

Ziheng Wei
School of Computer Science, Wuhan University
Wuhan, Hubei, China
ziheng.wei@whu.edu.cn
ORCID: 0000-0002-1859-0374

Sebastian Link
School of Computer Science, The University of Auckland
Auckland, New Zealand
s.link@auckland.ac.nz
ORCID: 0000-0002-1816-2863

Abstract*—Applications often need a key that holds on part of a table only. An e-mail address must be unique among the accounts that are not deleted, and a product code among the current versions. Applications declare such keys as filtered (partial) unique indexes, which make up one in nine unique indexes in ten open-source application schemas. Query optimisers could use these keys, as they use ordinary ones, to remove redundant joins and `DISTINCT`s. In practice they do not trust them. PostgreSQL did, returned wrong results, and has ignored them since 2023. CockroachDB and SQL Server ignore them too, and SQLite only matches predicates word for word. We show when trust is sound. A proof of uniqueness may use only the table's own restrictions, and it must use the `null` semantics of the rewrite it serves. Rewrites shaped like joins need uniqueness with `null`s distinct, and rewrites shaped like groupings need it with `null`s not distinct. Under two side conditions, the first question can be decided in polynomial time. The second is coNP-complete. Yet nearly all implications between the indexes of real schemas are easy. Our prover combines all unique indexes and `CHECK` constraints of a table and carries uniqueness through joins. It fits into PostgreSQL 18.6 with 38 changed planner lines, and into CockroachDB. It changed no result on 1,000,000 random queries. On catalogs over 226 real tables, it allows exactly the rewrites that the catalogs imply, where stock PostgreSQL allows none. It adds 0.03 ms to planning. Users' queries over Stack Exchange run up to $3.3\times$ faster, mostly through keys carried through joins, and queries we wrote over MusicBrainz run up to $4.9\times$ faster.

***Index Terms*—query optimisation, filtered unique indexes, partial indexes, keys, null markers, implication**

I. Introduction

Keys that hold on part of a table: Many business rules are keys with an exception. A user's e-mail address must be unique among the accounts that are not deleted. A deleted account may keep its address, and a new account may then reuse it. An item identifier must be unique among the current versions of a slowly changing dimension, while old versions repeat it. A template name must be unique within one account, one inbox, or the whole installation, depending on its scope. SQL's `UNIQUE` constraint always holds on the whole table, so it cannot state any of these rules. Database systems offer *filtered unique indexes* instead. Such an index enforces uniqueness only over the rows that satisfy its predicate:

```
CREATE UNIQUE INDEX ON users (email) WHERE
        deleted_at IS NULL;
```

PostgreSQL and SQLite call them partial indexes [50], [41], [45], and SQL Server calls them filtered indexes [46]. Frameworks such as Rails and Django generate them from model declarations. In ten open-source application schemas we found 174 of them among 1,513 unique indexes, and seven of the ten applications use them (Section II).

What an optimiser does with keys: A unique index is an integrity rule, but it is also a fact that the optimiser can use. Object-relational mappers, reporting tools and views often generate queries with work that a key makes redundant. Optimisers remove this work with a family of *uniqueness-based rewrites* [44], [47]:

- *Left-join removal.* A view may left-join an optional table to show its columns. Suppose a query does not use these columns, and the joined table has at most one match for each outer row. Then the join changes nothing, and it can be dropped.
- *Semi-join to join.* A predicate `x IN (SELECT y ...)` can be planned as an ordinary join only after duplicate values of `y` are removed. A join gives the optimiser more freedom in the join order. If `y` is a key, there are no duplicates to remove.
- *`DISTINCT` and `GROUP BY` removal.* If the selected or grouping columns contain a key, every row is already distinct, and every group has one row.
- *Inner-unique joins.* If each outer row has at most one partner, the join can stop searching after the first match. The cost model can also take this into account.

All four systems we study apply some of these rewrites with ordinary unique indexes. With filtered ones, the picture is different (Section II-B):

- *PostgreSQL* once used a partial unique index whenever the query's clauses implied its predicate. This led to wrong results (bug #17975). Since June 2023, every supported version ignores partial indexes in all of these rewrites [42], even when the query's predicate *is* the index's predicate. Users have asked for the rewrites back [14]. A patch proposed in 2026 addresses estimates only [15].
- *SQLite* removes a left join over a partial unique index only if the query's terms match the index predicate word

for word. For example, the query term `st = 0` does not match the index predicate `st IN (0, 1)`. SQLite never removes `DISTINCT` with a partial index.
- *CockroachDB* and *SQL Server* use filtered unique indexes for none of these rewrites.

Two pitfalls: The systems have good reason to be careful. A filtered key holds only on its scope. A rewrite that relies on the key is sound only if every row that the rewrite reasons about lies in that scope. There are two ways to get this wrong, and both have happened.

The first concerns *which conditions may establish the scope*. Take the index `UNIQUE (k) WHERE c IS NOT NULL` on a table `t`, and the query `o JOIN t ON o.k = t.k JOIN u ON t.c = u.c`. The second join keeps only the rows of `t` whose `c` is not `null`. So it seems that `t` is unique on `k`. But the first join is evaluated before the second, on all rows of `t`. If the planner marks the first join as unique, it stops at the first match on `k`. That match may have `null` in `c`, and the second join then discards it. The row that would have survived is never found. This is PostgreSQL's bug #17975. The lesson is that a uniqueness proof must hold for the rows of a table *before* any join. It may therefore use only the table's own restrictions.

The second pitfall concerns *null*. By default, a SQL unique index allows any number of rows with `null` in the key, because `null` does not equal `null`. A join on the key never matches `null`. Such an index therefore still proves that a join finds at most one partner. But `DISTINCT` and `GROUP BY` put all `null`s into one group. The same index therefore does not prove that `DISTINCT` is redundant: two rows with a `null` key form one group, not two.

Both pitfalls have measurable effects. On random schemas, a planner that proves an index's predicate from another join's clause, as PostgreSQL did, changes 442 of 20,000 query results. On 217 open-data tables, a planner that ignores the `null` semantics would remove 22,171 `DISTINCT`s, and 3,121 of these removals, on 133 of the tables, would be wrong (Section VII).

Our approach: We read a filtered unique index as a constraint, which we call a *conditional UNIQUE*. The planner's question then becomes one of *implication* between constraints. Consider the unique indexes of a table, together with its `NOT NULL` columns and `CHECK` constraints. Do they guarantee, for every possible content of the table, that the rows matching the query's restriction are unique on the columns that the query joins or groups on? Implication concerns the declared constraints, not the current data, so its answer stays correct when the data changes.

Implication also does more than match the query against one index at a time. Several indexes can together guarantee a key that none of them states alone. A query's scope can differ from an index's scope and still lie within it. A `CHECK` constraint can exclude the rows that an index does not cover. Fig. 1 shows a query whose `DISTINCT` is redundant only because two filtered keys are combined across a join.

`users`

id	email	deleted_at
1	a@x	null
2	a@x	2024-03-01
3	b@x	null

`user_emails`

user_id	address	is_primary
1	a@x	true
1	a2@y	false
3	b@x	true

```
UNIQUE (email) WHERE deleted_at IS NULL
  {⟨deleted_at: ⊥⟩} : U(email), email NOT NULL
UNIQUE (user_id, is_primary) WHERE is_primary
  {⟨is_primary: true⟩} : U(user_id, is_primary)

SELECT DISTINCT u.email, e.address FROM users u
  JOIN user_emails e ON e.user_id = u.id
    AND e.is_primary
  WHERE u.deleted_at IS NULL
```

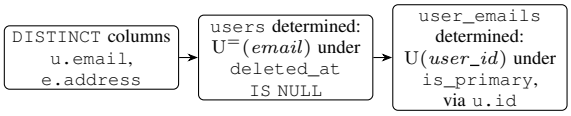


Fig. 1. Running example. Each live user has one e-mail address, and each user has one primary address. Hence every row of the join is determined by its `DISTINCT` values, and `DISTINCT` is redundant. Without `e.is_primary`, user 1 has two rows. Without `deleted_at IS NULL`, users 1 and 2 share `a@x`. If `email` were nullable, $\mathrm{U}(email)$ would not give $\mathrm{U}^{=}(email)$, because `DISTINCT` puts all `null`s into one group.

If we ask the question with the table's own restrictions only, we avoid the first pitfall. If we ask it with the `null` semantics that the rewrite needs, we avoid the second. The two semantics also differ in complexity. For join-shaped rewrites, implication can be decided in polynomial time when the predicates are conjunctive and have no inequalities or `IN`-lists on the compared columns. For group-shaped rewrites, implication is coNP-complete. Under the same conditions, it is hard only through nullable grouping columns that the query leaves open. Real schemas fall on the easy side. Among the indexes of the ten applications, 2,953 of 2,960 implications are as easy as in the first case. When an implication fails, a table of two rows shows why.

What we found: We built a prover based on this theory into the planner of PostgreSQL 18.6, with 1,683 new lines of C and 38 changed lines, and we ported it to CockroachDB.

- *It is sound.* On 1,000,000 random queries it changed 154,532 plans and no result. If any of its safeguards is removed, the random queries or a feature test detect the error.
- *It is exact on real catalogs.* We declared catalogs of filtered unique indexes over 226 real tables. The prover allows exactly the rewrites that each catalog implies, which is more than one index at a time allows. Stock PostgreSQL allows none.
- *It is cheap.* It adds 0.03 ms to planning in the median.
- *Its gains depend on who writes the queries.* Application code, transformation code and reports use filtered keys for integrity, and they rarely filter on the keys' scopes. Users who query Stack Exchange do. Their queries gain up to $3.3\times$, but mostly through ordinary keys carried through joins, which the prover adds to PostgreSQL. Views over versioned data lose every join they add. Queries we wrote

TABLE I
UNIQUE INDEXES IN TEN APPLICATION SCHEMAS

Application	Tables	Unique	Filtered	≥ 2 per table
GitLab	1,135	930	128	26
Mastodon	122	91	14	3
Forem	131	110	12	4
Chatwoot	109	93	9	2
Loomio	76	68	5	1
DocuSeal	44	36	3	0
Maybe	59	30	3	1
Three others*	323	155	0	0
Total	1,999	1,513	174	37

*Campsite, Consul, Open Food Network

over MusicBrainz gain up to $4.9\times$.

Contributions:

1) *The state of practice.* We study filtered unique indexes in ten application schemas, and what PostgreSQL, SQLite, CockroachDB and SQL Server do with them (Section II).
2) *The complexity of the planner's question.* For conditional `UNIQUE` constraints under both `null` semantics, implication is polynomial for join-shaped rewrites under two side conditions. It is coNP-complete for group-shaped rewrites, but fixed-parameter tractable under the same conditions. It is coNP-complete whenever a side condition fails (Sections III and IV). A finite axiomatisation is in the appendix.
3) *A sound prover* in PostgreSQL 18.6. It is exact for one relation over the indexes it translates. It serves the existing join rewrites and the reduction of `GROUP BY`, adds the removal of `DISTINCT`, reads `CHECK` constraints, and carries both semantics through inner, left, semi- and anti-joins. We also port it to CockroachDB (Section V).
4) *Evidence* of soundness, of exactness against four systems on real catalogs, of cost, and of gains on workloads grouped by who writes the queries (Section VII).

The appendix contains the proofs, the axiomatisation, a recommender for filtered unique indexes, and further experiments.

II. FILTERED UNIQUE INDEXES IN PRACTICE

A. In application schemas

We took the schemas of ten open-source applications that use PostgreSQL: GitLab, Mastodon, Forem, Chatwoot, Loomio, Consul, Open Food Network, Maybe, DocuSeal and Campsite. For GitLab we read its `structure.sql`, and for the others their Rails `schema.rb`. We parsed their unique indexes, primary keys, `UNIQUE` constraints and `NOT NULL` columns. GitLab keeps a copy of some indexes for each partition, and we count these copies once. Table I summarises the schemas.

The predicates fall into a few shapes:

- *Excluding `null` keys* (87), as in `UNIQUE (k) WHERE k IS NOT NULL`. All 14 of Mastodon's filtered indexes and 63 of GitLab's have this shape. Under `NULLS DISTINCT` the predicate changes nothing, so such an index states an ordinary key. The predicate only keeps the index small, or it is a habit from SQL Server, where it is the only way to allow several `null`s.
- *A constant or flag* (33): `status = 0`, `published IS TRUE`, `rule_type = 2`.
- *A missing value in another column* (26): `ended_at IS NULL`, `resolved_at IS NULL`. These express soft deletion, current versions and open episodes.
- *IN-lists* (9): `status = ANY (ARRAY[0, 1])`.
- *Conjunctions of the above* (5). For example, Chatwoot has three scopes for e-mail templates: one per installation (`account_id IS NULL AND inbox_id IS NULL`), one per account (`account_id IS NOT NULL AND inbox_id IS NULL`), and one per inbox.
- *Inequalities* (14): `package_type = 7 AND status <> 4`, `custom_domain <> ''`.

Six indexes have keys that are expressions, such as `lower(email)`. The other 168 have predicates built from `=`, `<>`, `IN`, `IS [NOT] NULL`, `AND` and `OR`. In the terms of Section III, all 168 are conditions with one template: 159 are conjunctions, and 9 are `IN`-lists. SQL writes an `IN`-list as a disjunction, but one template entry can hold it. In the 37 tables with two or more filtered unique indexes, a planner that looks at one index at a time can miss what the indexes say together. Only two filtered indexes are implied by others, in both cases by the primary key. Developers thus do not declare filtered indexes redundantly. Beyond these applications, we looked at the SQL files of SchemaPile [7], which holds 22,989 permissively licensed schemas from GitHub. There, filtered indexes make up 12.9% of the `CREATE UNIQUE INDEX` statements in SQL Server files (118 of 913) and 5.4% in PostgreSQL files (182 of 3,362). Their predicates have the same shapes: constants and flags, `null` tests on other columns, and the exclusion of `null` keys. Of the 356 filtered unique indexes in all dialects, 69 use conjunctions, only 8 use `OR`, and 6 use range comparisons.

In analytics code: Data warehouses built with dbt state keys as tests. In the two largest public dbt projects we found, 38 of 369 uniqueness tests have a scope (`where:`). In 27 of them, the scope expresses a documented exception to the key.

In open data: MusicBrainz publishes its PostgreSQL schema and data, with 17 filtered unique indexes. On 13 alias tables, `(entity, locale) WHERE primary_for_locale AND locale IS NOT NULL` allows at most one primary alias per entity and locale. Artists and labels are unique by name, and a comment tells namesakes apart: `(name) WHERE comment IS NULL` and `(name, comment) WHERE comment IS NOT NULL`. This pair is the encoding of $\mathrm{U}^{=}()$ in Proposition 3. Since `comment` became `NOT NULL`, the second index states an ordinary key, which PostgreSQL ignores because the index is partial. Every key holds in the data of 28 October 2026.

TABLE II
UNIQUENESS-BASED OPTIMISATIONS WITH A PLAIN AND A FILTERED UNIQUE INDEX WHOSE PREDICATE EQUALS THE QUERY PREDICATE

System	Optimisation	plain	filtered
PostgreSQL 18	left-join removal	yes	no
	semi-join to join	yes	no
	inner-unique join	yes	no
	`GROUP BY` column removal	yes	no
	self-join elimination	yes	no
	`DISTINCT` removal	no	no
SQLite 3.54	`DISTINCT` removal	yes	no
	left-join omission	yes	syntactic
CockroachDB	left-join removal	yes	no
	`DISTINCT` removal	yes	no
	`GROUP BY` reduction	yes	no
SQL Server	left-join removal	yes	no
	semi-join to join	yes	no
	`DISTINCT` removal	yes	no
	`GROUP BY` removal	yes	no

B. In optimisers

Table II lists the uniqueness-based optimisations of PostgreSQL 18.6, SQLite 3.54, CockroachDB 26.3 and SQL Server 2025. For each, it shows whether a filtered unique index enables it. We tested each system with an ordinary unique index on k and a partial unique index on k `WHERE d IS NULL`, using queries with exactly that predicate. We then traced each behaviour to the source code.

PostgreSQL: Before planning, `check_index_predicates` sets the `predOK` flag of a partial index if `predicate_implied_by` proves the index's predicate from the relation's restrictions. Until 2023, `relation_has_unique_index_for` accepted partial indexes with `predOK` set. This function is the uniqueness test behind left-join removal, semi-join reduction and inner-unique joins. But `predOK` may be proved from join clauses that are evaluated only after the inner relation is read. The planner could therefore treat a relation as unique when it is not. The fix (commit `7fcd7ef2a9`, back-patched to all supported versions) excludes every index with a predicate from `relation_has_unique_index_for` and `rel_supports_distinctness`. Version 18 added the reduction of `GROUP BY` by unique indexes and the elimination of self-joins, and both skip partial indexes from the start. Partial indexes with `predOK` now matter only for the estimated number of distinct values. PostgreSQL removes `DISTINCT` for no base table, filtered or not.

SQLite: The function `isDistinctRedundant` skips partial indexes. Left-join omission relies on a flag that marks the access path of the inner table as returning at most one row. A partial unique index sets this flag when `whereUsablePartialIndex` finds, for each conjunct of the predicate, a term of the `ON` or `WHERE` clause on that table that implies it. The test, `sqlite3ExprImpliesExpr`, accepts the same expression, one branch of an `OR`, or `IS NOT NULL` after a comparison that rejects `null`. This is sound but fragile. Both `b.st = 0` against `WHERE st IN (0, 1)` and `NOT (b.d IS NOT NULL)` against `WHERE d IS NULL` keep the join, and two indexes are never combined.

CockroachDB: The optimiser derives a table's keys in `MakeTableFuncDep`. This function skips partial unique indexes and partial `UNIQUE WITHOUT INDEX` constraints, because their keys hold on part of the table only. It does add such a key to a scan *of* the partial index. But that scan is an alternative found during exploration, while join elimination, `DISTINCT` removal and grouping reduction are normalisation rules applied to the table's canonical scan. None of these rules used a partial key in our tests. For example, `DISTINCT k` stays above a scan of the partial unique index on `k`, even for `UNIQUE (k) WHERE k IS NOT NULL`.

SQL Server: Filtered unique indexes have conjunctive predicates of comparisons and `IN`-lists, and their `UNIQUE` treats `null` as a value [46]. Range comparisons are outside our language, so an index that uses one cannot serve as a premise. White [52] shows that the optimiser costs a one-to-one join over filtered unique indexes as many-to-many, and he recommends redundant predicates or indexed views as workarounds. We tested SQL Server 2025 (RTM-CU10, Developer edition) in the same way as the other systems. With a unique constraint, it removes the left join and `DISTINCT`, reduces `GROUP BY`, and turns the semi-join into a join. With a filtered unique index on the query's own predicate, it does none of these. It even keeps the aggregate for `DISTINCT k` when it reads the filtered index `UNIQUE (k) WHERE k IS NOT NULL`.

III. CONDITIONAL UNIQUE

A table schema is a finite set R of columns, with a subset $NN \subseteq R$ of columns declared `NOT NULL`. A *table* over R is a finite multiset of records. A record maps each column to a value or to `null`, and it has no `null` on a column of NN. As in SQL, a table may contain duplicate records.

Predicates as templates: We bring index predicates and query restrictions into one normal form, so that we can reason about both alike. A *template* τ maps each column to an entry. The entries are $\square$ (anything), $\top$ (`IS NOT NULL`), $\bot$ (`IS NULL`), $\in S$ (`= c` or `IN (S)`), and $\notin S$ (`<> c` or `NOT IN (S)`), for a finite non-empty set S of constants. A `null` matches only $\square$ and $\bot$, because a comparison with `null` is never true in a `WHERE` clause. A value v matches $\square$ and $\top$. It matches $\in S$ if $v \in S$, and $\notin S$ if $v \notin S$. A record *matches* τ if it matches every entry. A *condition* is a finite set of templates, and a record matches it if the record matches one of them. Every predicate built from =, <>, `IN`, `NOT IN`, `IS [NOT] NULL`, `AND` and `OR` over columns and constants is a condition. To see this, bring the predicate into disjunctive normal form. Two entries on the same column combine into one entry, or into a contradiction. A conjunctive predicate is a condition with one template. Range comparisons are outside the language. We write $\langle d{:}\,\bot\rangle$ for the template with $\bot$ on d and $\square$ elsewhere.

Definition 1. A *conditional* `UNIQUE` *constraint* has the form $C : \mathrm{U}(K)$ or $C : \mathrm{U}^{=}(K)$ for a condition C and a set $K \subseteq R$. A table r satisfies it if no two records of r, at different positions, both match C and agree on K. For $\mathrm{U}(K)$ (`NULLS DISTINCT`), two records agree on K if they have the same value on every column of K. For $\mathrm{U}^{=}(K)$ (`NULLS NOT DISTINCT`), `null` also counts as equal to `null`.

The filtered unique index of the introduction is $\{\langle \mathit{deleted_at}\colon \bot\rangle\} : \mathrm{U}(\mathit{email})$. In PostgreSQL and SQLite, `CREATE UNIQUE INDEX ... WHERE P` states $C_P : \mathrm{U}(K)$. PostgreSQL's `NULLS NOT DISTINCT` variant and SQL Server's filtered unique index state $C_P : \mathrm{U}^{=}(K)$. A primary key is $\{\langle\rangle\} : \mathrm{U}(K)$ with $K \subseteq NN$. A key over no columns, $C : \mathrm{U}(\emptyset)$, says that at most one record matches C. PostgreSQL can state it as a unique index on a constant expression.

Lemma 2 (Pairs)**.** *A set Σ of conditional `UNIQUE` constraints implies φ iff every table of at most two records that satisfies Σ satisfies φ.*

Proof. Violations are witnessed by two records, and every constraint holds on a sub-table of a table that satisfies it. □

Between the dialects: Each `null` semantics can be expressed by the other, which matters when a schema moves between systems. For a nullable key column, $\mathrm{U}()$ is $\mathrm{U}^{=}()$ restricted to the records without `null` on K. SQL Server users write this as `WHERE k IS NOT NULL`. In the other direction, SQLite and PostgreSQL before version 15 can express $\mathrm{U}^{=}()$ only with one index for each pattern of `null`s on the key:

Proposition 3 (Translation)**.** *Let F be the nullable columns of K. Then*

$$C : \mathrm{U}(K) \equiv C \wedge \langle F\colon \top\rangle : \mathrm{U}^{=}(K),$$
$$C : \mathrm{U}^{=}(K) \equiv \{\, C \wedge p_N : \mathrm{U}(K \setminus N) \;:\; N \subseteq F \,\},$$

where p_N has $\bot$ on N and $\top$ on $F \setminus N$, and $C \wedge p$ refines every template of C by p (dropping contradictory ones). The second equivalence uses $2^{|F|}$ constraints, one per pattern of `null`s on the nullable key columns.

Proof. By Lemma 2, it suffices to compare which pairs of records the two sides forbid. Consider the first equivalence. A pair that matches C and agrees on K with values has no `null` on K. So it matches $C \wedge \langle F\colon \top\rangle$, and it agrees on K with `null` equal to `null`. Conversely, a pair that matches $C \wedge \langle F\colon \top\rangle$ has no `null` on F, and none on $K \setminus F \subseteq NN$. For such a pair, agreeing on K with `null` equal to `null` is the same as agreeing with values. Now consider the second equivalence. Suppose two records match C and agree on K with `null` equal to `null`. On each column of K, they are both `null` or carry the same value. So they have the same set $N \subseteq F$ of `null` columns on K, they both match $C \wedge p_N$, and they agree on $K \setminus N$ with values. They therefore violate the translated constraint for N. Conversely, two records that match $C \wedge p_N$ and agree on $K \setminus N$ with values are `null` on N in both. So they match C, and they agree on K with `null` equal to `null`. □

It is open whether fewer than $2^{|F|}$ constraints with `null`s distinct can express $C : \mathrm{U}^{=}(K)$ in general. We checked both equivalences on 5,000 random instances.

Conditional `UNIQUE` strictly generalises the embedded uniqueness constraints of [54]. These are the case $\{\langle E\colon \top\rangle\} : \mathrm{U}(U)$ with $U \subseteq E$, and no set of them is equivalent to $\{\langle d\colon \bot\rangle\} : \mathrm{U}(k)$ or to $\{\langle s\colon \in\{c\}\rangle\} : \mathrm{U}(k)$. Conditional `UNIQUE` is incomparable with conditional functional dependencies [28]. Two identical records satisfy every CFD, but they violate every conditional `UNIQUE` whose scope contains them. A table with one record satisfies every conditional `UNIQUE`, but not every CFD (Appendix A).

IV. Reasoning

Every rewrite of Section V asks the same question. We are given the filtered unique indexes Σ of a table, its `NOT NULL` columns, and the restriction C that a query block places on the table. Does every table that satisfies Σ have at most one record per value of the columns K, among the records that match C? Which equality is meant depends on the rewrite:

- *Join-shaped* rewrites compare K with an equality predicate, which never matches `null`. They are left-join removal, semi-join to join, inner-unique joins and self-join elimination. They need $\Sigma \models C : \mathrm{U}(K)$.
- *Group-shaped* rewrites put all `null`s into one group. They remove `DISTINCT`, `GROUP BY`, or grouping columns. They need $\Sigma \models C : \mathrm{U}^{=}(K)$.

A planner that uses a filtered index naively answers the first question when the rewrite asks the second. This is the `null` pitfall of the introduction. The rewrite also determines the complexity, as we show below.

Example 4. Premises of both semantics can combine. Let k be nullable. The constraint $\{\langle\rangle\} : \mathrm{U}(k)$ alone does not imply $\{\langle\rangle\} : \mathrm{U}^{=}(k)$, because it allows two rows with `null` in k. A `DISTINCT k` therefore cannot be removed. Now add $\{\langle k\colon \bot\rangle\} : \mathrm{U}(\emptyset)$, which allows at most one row with `null` in k. In PostgreSQL this is a unique index on a constant `WHERE k IS NULL`, an expression key that our prover does not read. Together the two constraints imply $\{\langle\rangle\} : \mathrm{U}^{=}(k)$. Two rows that agree on k with `null` equal to `null` either share a value, which the first index forbids, or are both `null`, which the second forbids. On the other hand, scopes do not combine by union. The constraints $\{\langle d\colon \bot\rangle\} : \mathrm{U}^{=}(k)$ and $\{\langle d\colon \top\rangle\} : \mathrm{U}(k)$ cover every row. Yet they imply neither $\{\langle x\colon \in\{1\}\rangle\} : \mathrm{U}(k)$ nor $\{\langle x\colon \in\{1\}\rangle\} : \mathrm{U}^{=}(k)$. Two rows with $x = 1$ and the same k, one with `null` in d and one without, lie in no common scope.

The idea: By Lemma 2, an implication fails exactly when two records satisfy every premise and violate the goal. The search for such a pair is easy when each column can be treated on its own. A premise is defeated if one of the two records

lies outside its scope, or if the records differ on its key. For each column, we pick the values of the two records that defeat as many premises as possible. If one choice is best for every premise at once, the pair made of these choices settles the question. The side conditions of Theorem 5 are exactly what makes one choice per column best. This choice is a `null` in one record and a value of its own in the other, or two different values. Inequalities in the premises and `IN`-lists in the goal break this, because the best choice then depends on the premise.

Call a column *value-forced* for templates τ_i, τ_j of the goal if every pair that violates $C : \mathrm{U}(K)$, with t_1 matching τ_i and t_2 matching τ_j, carries a value on it in both records. These are the columns of K and of NN, and the columns on which both templates have $\top$, $\in S$ or $\notin S$.

Theorem 5 (Join-shaped rewrites)**.** *Let every constraint of Σ have a one-template condition (any mix of $\mathrm{U}()$ and $\mathrm{U}^{=}()$), and let C be a condition such that, for all $\tau_i, \tau_j \in C$, no constraint of Σ has a $\notin$-entry on a value-forced column and neither template has an entry $\in S$ with $|S| \geq 2$ on such a column. Whether $\Sigma \models C : \mathrm{U}(K)$ is decidable in time $O(|C|^2 \cdot |\Sigma| \cdot |R|)$.*

Proof. By Lemma 2, $\Sigma \not\models C : \mathrm{U}(K)$ iff some pair t_1, t_2 satisfies Σ, where t_1 matches a template $\tau_i \in C$, t_2 matches a template $\tau_j \in C$, and the two records are equal and free of `null` on K. A pair satisfies a one-template constraint $\sigma = \{\rho\} : \mathrm{U}(K_\sigma)$ iff some column A *kills* σ. This means that $t_1[A]$ or $t_2[A]$ fails $\rho(A)$, or that $A \in K_\sigma$ and $t_1[A], t_2[A]$ are not equal values. For $\mathrm{U}^{=}()$, they must also not both be `null`. Killing is thus a disjunction over columns, and what a column A kills depends only on $(t_1[A], t_2[A])$. Call $(t_1[A], t_2[A])$ *admissible* if it matches $\tau_i(A)$ and $\tau_j(A)$, respects NN, and consists of two equal values when $A \in K$. We claim that on every column, one admissible state kills everything that any admissible state kills. The pair made of these states then satisfies Σ iff some admissible pair does.

Let a *fresh* value be one that no constraint mentions. Take first a column $A \notin K$ on which one record may be `null` and the other a value. A `null` and a fresh value fail every entry other than $\square$. (If a template forces the value to a constant, we use that constant.) The `null` fails $\top$, $\in S$ and $\notin S$, and the value fails $\bot$ and $\in S$. The two records also differ on A under both semantics. Otherwise the column is value-forced or forced to `null`. On a value-forced column, two different values fail $\bot$ and every $\in S$ that a fresh value fails, and they separate the records. The values are fresh unless the templates fix them to single constants. No constraint has a $\notin$-entry on such a column, so nothing else can be killed there. Finally, for $A \in K$ the records share a value. This value is fresh unless a template fixes it to a single constant, and again it kills everything that a value can kill on A. Building and checking the $|C|^2$ pairs takes the stated time. $\square$

Theorem 6 (Group-shaped rewrites)**.** *Deciding $\Sigma \models C : \mathrm{U}^{=}(K)$ is coNP-complete, already when C and all constraints of Σ have one template each. Under the side conditions of Theorem 5 it is decidable in time $O(2^b \cdot |C|^2 \cdot |\Sigma| \cdot |R|)$, where b is the number of columns of K that are neither `NOT NULL` nor fixed to $\top$, $\bot$ or a constant by C.*

Proof. Membership in coNP holds because a violating pair can be guessed and checked (Lemma 2). For the bound, consider a pair that violates $C : \mathrm{U}^{=}(K)$. On each of the b open key columns, the two records are either both `null` or share a value. These two states kill incomparable sets: the first kills $\top$, constants and $\mathrm{U}()$-keys on that column, and the second kills $\bot$. Every other column behaves as in Theorem 5. Hence 2^b canonical pairs per pair of templates suffice.

For hardness, we reduce from CNF satisfiability over variables $x_1, \ldots, x_n$. Take nullable columns $A_1, \ldots, A_n$, a column $B \in NN$, and the goal $\{\langle\rangle\} : \mathrm{U}^{=}(A_1 \cdots A_n B)$. For each clause, add one constraint $\{\rho\} : \mathrm{U}(B)$, where $\rho(A_i) = \bot$ if x_i occurs positively in the clause, $\rho(A_i) = \top$ if it occurs negatively, and $\rho(A_i) = \square$ otherwise. A pair that violates the goal agrees on every A_i. Either both records are `null` there, which we read as "x_i is false", or they share a value, which we read as "true". The pair satisfies a clause's constraint iff some literal kills it, that is, iff the assignment satisfies the clause. Hence $\Sigma \models \varphi$ iff the CNF is unsatisfiable. Since $B \in NN$, the constraints are equivalent to $\mathrm{U}^{=}(B)$, so hardness holds in SQL Server's semantics alone. They also meet the side conditions of Theorem 5. Without these conditions, b does not bound the search: Theorem 7 below is hard with $K \subseteq NN$, where $b = 0$. $\square$

Neither side condition of Theorem 5 can be dropped, on key columns or elsewhere. The only case left open is `IN`-lists of two constants outside the key.

Theorem 7 (Inequalities and IN-lists)**.** *Deciding $\Sigma \models \{\tau\} : \mathrm{U}(K)$ is coNP-complete for one-template conditions when constraints of Σ may have $\notin$-entries on columns of K or on other value-forced columns, and also when τ may have entries $\in S$ with $|S| = 2$ on columns of K or $|S| = 3$ on other value-forced columns.*

The cases on K reduce from CNF satisfiability as Theorem 6 does (Appendix A). The cases outside K reduce from 3-colourability: take `NOT NULL` columns A_u, one per vertex, and B, the goal $\{\langle\rangle\} : \mathrm{U}(B)$, and for every edge uv and colour c the constraint $\{\langle A_u{:}\,\notin\{c\}, A_v{:}\,\notin\{c\}\rangle\} : \mathrm{U}(B)$ (for $\in$: τ has $\in\{1,2,3\}$ on every A_u, and the constraint $\in\{1,2,3\} \setminus \{c\}$). A pair violating the goal satisfies the constraint for uv and c iff c is among its four values on A_u and A_v. Given a proper colouring, let both records carry on A_u the two colours other than that of u: every edge then shows all three. Conversely, two records show at most two colours per vertex. Colour each vertex with one that its records miss. Then no edge is monochromatic, because its endpoints together show every colour. So $\Sigma \models \{\tau\} : \mathrm{U}(B)$ iff the graph is not 3-colourable. We checked both reductions by brute force on 240 random graphs.

The general procedure: Without the side conditions, the argument of Theorem 5 still works column by column.

Whether a pair satisfies Σ is a monotone function of features of each column, such as "a record fails this entry" or "the records differ on this key column". On each column, therefore, only the states with a maximal set of features matter. The values worth trying are the constants mentioned on the column, two fresh values, and `null`. Let μ_A be the number of maximal states on column A. Then implication is decided in time $O(|C|^2 \cdot \prod_A \mu_A \cdot |\Sigma| \cdot |R|)$. Theorems 5 and 6 are the cases $\mu_A = 1$ and $\mu_A \leq 2$. For indexes whose predicates have several templates, that is, predicates with `OR`, the features are kept for each record separately. Implication is then coNP-complete for both semantics, already for a single premise. A record escapes such a scope only by failing every template, and this encodes clauses.

We implemented this procedure for the full language. It agrees with exhaustive search over all pairs on 3,000 random instances with `IN`, `NOT IN`, both semantics and up to two templates per condition. It also agrees with a SAT encoding of the existence of a violating pair on 3,000 instances without $\notin$. Under the side conditions of Theorem 5, no column had two maximal states in 10,640 random instances. We checked Theorems 5–7 against SAT on 40,000 random instances, and the reductions on 700 random 3-CNFs.

Axiomatisation: Implication also has a finite axiomatisation. Its seven rules are sound and complete for the full language, under both `null` semantics and any mix of them (Appendix C). The completeness proof rests on a *cell lemma*. Fix the constants of an instance. Then every record has a *profile*, which gives for each column a constant, `null`, or "another value". Take any two profiles in the goal's scope. Among the pairs of records with these profiles, take the pair that agrees on the fewest columns. Σ implies the goal iff every such pair violates some member of Σ. The theory assumes infinite domains. For finite types such as `boolean`, every decision is still sound. But an implication that holds only because a type is finite can be missed.

Refusals have witnesses: A cell that no premise covers gives a counterexample of two records. This is a table that satisfies every unique index, `NOT NULL` and `CHECK` constraint of the relation, and that still has two rows in the query's scope that agree on the key. When our prover refuses a rewrite, its search ends at such a pair. It does not report the pair yet, but a planner could show it as the reason. On the real catalogs of Section VII-B, where the prover decides implication exactly, every rewrite it refuses has such a witness. For 167 `DISTINCT`s that a planner ignoring the `null` semantics would remove, the data itself contains one.

What the theory says to the planner: In the schemas of Section II-A, all 168 predicates are conditions with one template, and the hard cases do not arise. We took each of the 1,480 unique indexes that are not primary keys and have keys of plain columns. We used it as the goal, with the other unique indexes of its table as Σ, under both semantics. Of these 2,960 implications, 2,953 needed one canonical pair per pair of templates, as in the case of Theorem 5. This is an observation about the instances, not membership in that case, since the count includes all 34 implications on the six tables with `<>` predicates. Each of the other 7, all with $\mathrm{U}^{=}()$, had a single column with more than one maximal state. A decision took 0.04 ms in the median and at most 2.9 ms, in Python. Under the side conditions, group-shaped rewrites are hard only through nullable grouping columns that the query leaves open. For a `DISTINCT` over `NOT NULL` columns, or a `GROUP BY` whose block filters its columns with `IS NOT NULL` or equalities, $b = 0$. Finally, the 84 indexes on plain columns that only exclude `null` keys are recognised as the ordinary keys they are, which PostgreSQL ignores because they are partial.

V. A Uniqueness Prover for the Planner

Interface: PostgreSQL's planner asks for uniqueness in one function, `relation_has_unique_index_for(root, rel, clauses, ...)`. It also has a cheap pre-check, `rel_supports_distinctness`, and a pass that removes grouping columns, `remove_useless_groupby_columns`. Each of them asks whether a base relation `rel` is unique on the columns that `clauses` equate. Our prover replaces the line that skips partial indexes with a call $\textsc{Unique}(\Sigma_{rel}, NN_{rel}, C_{rel}, K, mode)$. Here Σ_{rel} is the set of all unique indexes of the relation whose predicates can be translated. C_{rel} is the condition of the relation's *base* restrictions (`baserestrictinfo`). It never includes join clauses, which removes the cause of bug #17975. The $mode$ is $\mathrm{U}()$ for the join-shaped callers and $\mathrm{U}^{=}()$ for the `GROUP BY` pass.

Translation: An index predicate is translated exactly, or the index is ignored. In the schemas of Section II-A, only the indexes with expression keys are ignored. Query restrictions may be translated with a larger scope than they have. A range comparison such as `x > 3`, or any other conjunct that the language cannot express, is dropped. This only makes the scope larger, so every answer stays sound. Reading such a conjunct as $\top$ on its column would not be sound, because `x IS DISTINCT FROM 3` admits `null`.

Decision: The prover runs the general procedure of Section IV. In the case of Theorem 5, this needs one canonical pair per pair of templates. Otherwise the prover searches the states of the columns depth first, keeping only maximal sets of features. It answers "not proven" after 10^5 nodes, which loses proofs but never soundness. On one relation, the prover is thus exact for the indexes it translates, unless the search gives up. The premise filter and the template limit described below, and the join rules, which are sufficient but not complete, lose further proofs. None of these losses affects soundness.

Rewrites: Table III lists the rewrites and their goals.

Implementation: We implemented the prover in PostgreSQL 18.6 as one new file of 1,683 lines of C and 38 changed lines in six planner files. Two settings, both off by default, control it: `enable_conditional_unique` turns the prover on, and `enable_conditional_unique_costing` also lets a

TABLE III
REWRITES AND THE CONSTRAINT THEY NEED ON THE RESTRICTED RELATION

Rewrite	Needs
Left-join removal (no inner column used)	$C : \mathrm{U}(K)$
IN/EXISTS semi-join to join	$C : \mathrm{U}(K)$
Inner-unique join†	$C : \mathrm{U}(K)$
Self-join elimination (not yet hooked)	$C : \mathrm{U}(K)$
DISTINCT removal*	$C : \mathrm{U}^{=}(K)$
GROUP BY or grouping-column removal*	$C : \mathrm{U}^{=}(K)$

* also over joins † only with enable_conditional_unique_costing

proof over a filtered index mark a join as Inner Unique. This changes physical choices. The executor stops the inner scan after the first match, and the planner costs the join as one-to-one. Before the data is cached, this can make queries slower (Section VII-D).

Where the prover is asked: When no unique index without a predicate settles a question, relation_has_unique_index_ext asks the prover. This one test serves left-join removal, inner-unique joins, semi-join reduction, and the removal of duplicates from IN subqueries. In addition, rel_supports_distinctness now accepts partial unique indexes.

A second pass over GROUP BY runs after the restriction clauses are distributed and before pathkeys are computed. It asks for a $\mathrm{U}^{=}()$ goal. It skips relations that an outer join can null, because a null-extended row agrees with any row whose key is null. It also uses keys without predicates whose columns the restriction may fix. For example, WHERE ns = 0 makes UNIQUE (ns, title) a key on title.

PostgreSQL removes no DISTINCT over a base relation, with or without a predicate, so we add this rewrite. Before the paths for DISTINCT are built, the prover is asked for a $\mathrm{U}^{=}()$ goal on the DISTINCT columns. This happens if the query reads a single base relation and has no DISTINCT ON, grouping, aggregates, window functions, set operations, or set-returning functions in its target list. A proof means that the rows reaching DISTINCT are already distinct, so the operator is dropped. DISTINCT items other than plain columns are ignored, because they can only make rows more distinct. In every case, the goal's condition comes from the relation's own restriction clauses, never from join clauses.

Safeguards in translation: Soundness rests on three rules. First, a conjunct that cannot be translated is dropped from the goal, which only enlarges its scope. It also makes an index unusable as a premise. Second, every equality must belong to the default btree operator family of the column's type, under a deterministic collation. Cross-type members of the family count too, such as int2 = int4, which ORMs generate for enum columns. An index may use any operator class with the same equality, for example text_pattern_ops. Third, constants are identified by the family's equality, not by how they are written. To keep the search small, templates are limited to 16. An index is a premise only if its key lies in the goal's key, or in the columns that the goal fixes to one constant or to null. Dropping premises loses proofs, but never soundness. Which indexes can serve as premises at all depends on catalog lookups but not on the question: an index qualifies if it has no expressions and uses the default equality on every key column. This set is therefore computed once per relation and planning run, and kept in a new field of RelOptInfo.

CHECK constraints: A validated and enforced CHECK constraint is a fact about every row. It therefore narrows the goal's scope, as a restriction does. A row passes a CHECK constraint c when c is true or null. The scope is therefore narrowed by the disjunction of c and "some column that c compares is null". A column that c only tests with IS [NOT] NULL or IS TRUE/FALSE cannot make c null. For example, MusicBrainz's CHECK ((locale IS NULL AND primary_for_locale IS FALSE) OR locale IS NOT NULL) gives every primary alias a locale. Constraints marked NOT VALID or NOT ENFORCED may be violated by old rows, so the prover ignores them.

Through joins: Uniqueness also carries through joins. Let G be the DISTINCT or GROUP BY columns. We call a relation *determined* by G if its part of every joined row is a function of the values of G. This part is one of its rows, or nulls. A relation is determined in two cases. In the first, its rows that pass its restrictions are unique, with nulls equal ($\mathrm{U}^{=}()$), on its columns in G that no outer join can null. In the second, they are unique ($\mathrm{U}()$) on columns equated with columns of determined relations. The equality must come from an equivalence class with two or more members, or from an outer join clause read from a determined outer side. Equality is strict, so $\mathrm{U}()$ suffices in the second case.

Some relations need special rules. Inner sides of semi- and anti-joins add no columns to a joined row. They need not be determined, but their columns determine nothing. A block with a full join is left alone. A relation on the nullable side of a left join is determined only once its whole outer side is determined. The reason is that the ON clause decides whether a row is null-extended, and it may read any outer column. The planner builds equivalence classes with a single member for sort keys, and these equate nothing.

If every other relation is determined, DISTINCT is dropped, and so is a GROUP BY without aggregates. With aggregates, the GROUP BY columns of a relation that the other columns determine are dropped, as long as one column remains. The pass also uses ordinary keys, which stock PostgreSQL uses for GROUP BY within one relation only.

Lemma 8 (Determination)**.** *Let the relations of a query block be joined by inner, left, semi- and anti-joins, and let the database satisfy the unique indexes, NOT NULL and validated CHECK constraints of every relation. If a relation S is determined by G under the rules above, then any two rows of the join that agree on G (null equal to null) agree on the part of S: the same row of S, or nulls in both. If every relation*

outside the inner sides of semi- and anti-joins is determined, no two rows of the join agree on G.

The proof is by induction on the order in which relations become determined. Each rule supplies one step, and the side conditions are exactly what the step needs (Appendix B). Deriving the keys of a join from those of its inputs is classical [44], [21], [22]. Filtered keys add two things. The premise of each step holds only under the relation's own restrictions. The semantics of the step also changes along the way: $\mathrm{U}^{=}()$ where a relation is entered through the grouping columns, $\mathrm{U}()$ where it is entered through an equality. The planner also has traps of its own. Our differential testing caught two errors in our first version: we read single-member classes as equalities, and we entered an outer join from a partly known outer side. TPC-DS caught a third: we read the columns of a semi-join as fixed values. Section VII-A reports how the final version and these mutants fare.

In a second optimiser: The prover is not tied to PostgreSQL. CockroachDB's optimiser removes joins, `DISTINCT` and grouping columns, and carries keys through joins. It does all of this from the functional dependencies of each expression, and it leaves partial unique indexes out of them (Section II-B). We ported the prover to Go, in 793 lines, on CockroachDB 26.2.9, the newest release in its public repository, whose stock optimiser behaves like 26.3 in our tests. Behind a session setting, we call the prover where the dependencies of a filter over a table scan are built. This takes 4 changed lines in the optimiser and 31 for the setting. A key that the prover proves is added as strict if it holds with `null`s equal, and as lax if it holds only with `null`s distinct. CockroachDB already distinguishes these two kinds. Filters that read another table stay in the join, so the prover sees base restrictions only. One gap is CockroachDB's own. A lax key of an inner relation does not become a strict key of a join whose equality makes it non-null. The running example therefore keeps its `DISTINCT` unless `user_id` is `NOT NULL`.

VI. Recommending Filtered Unique Indexes

Some filtered unique indexes are enforced only in application code, and others are declared by nobody. Such indexes can be recommended. A recommendation consists of a condition and a key. Conditions are cheap to propose, and they come from two sources. The schema proposes `IS [NOT] NULL` on nullable columns, boolean values, and the values of columns that look like a status, type or kind. The workload proposes the atoms that its queries apply to the table. Keys under a condition come from the data, and there a reduction does the work.

Proposition 9 (Reduction)**.** *For a condition C, the minimal keys K such that $C : \mathrm{U}(K)$ (respectively $C : \mathrm{U}^{=}(K)$) holds on a table r are the minimal unique column combinations of the records of r that match C, after each `null` is replaced by a value of its own (respectively all `null`s by one value).*

Any algorithm for unique column combinations [43], [3] therefore discovers conditional keys, and the complexity carries over. Whether a key of at most k columns exists is NP-complete, and W[2]-complete in k, already for the empty condition [4]. For fixed a, there are polynomially many conditions of at most a atoms. Without such a bound, the problem becomes trivial in an unhelpful way. Conditions over the constants of r can single out any set of rows. So the largest scope on which a key is unique is found by keeping one row per value of the key. In practice, the difficulty is not the search but *accidental* keys, which hold on the data but not by design. A recommender should prefer conditions that the workload applies, and keys that keep holding as the data changes. Appendix D gives the theory and two miners. One fixes the condition and follows the hybrid scheme of [43]. The other fixes the key and finds all valid conditions of at most two atoms, in one pass over the pairs of records that agree on the key.

VII. Evaluation

We ask five questions. Is the prover sound (Section VII-A)? Which rewrites does it find that the systems miss (Section VII-B)? What does it cost (Section VII-C)? What does it gain (Section VII-D)? And where do the keys that we declare on Stack Exchange and TPC-DS come from (Section VII-E)? All runs used one core of an Apple M5 Max with 36 GB of memory. PostgreSQL ran with default settings unless stated otherwise.

A. Soundness

Regression and feature tests: With the setting off, all 231 regression tests of PostgreSQL 18.6 pass, and only the output that lists the settings changes. With the setting on for every test, no result changes. The pass through joins drops 11 `GROUP BY`s and `DISTINCT`s in the tests of aggregation and join removal, all of them correctly. With the `Inner Unique` setting on as well, one more plan changes. This test checks that the partial index `UNIQUE (id2) WHERE id2 IS NOT NULL` does not make an inner join unique. The join now gets `Inner Unique`, which is correct, because a join on `id2` admits only values that are not `null`.

Our own 35 feature tests compare plans and results with the setting off and on. One test has the shape of bug #17975, a predicate that holds only through a join clause, and there the join is kept. In another, the query's scope is the union of two scopes, one per status, and the join is kept as well. The tests also run the example of Fig. 1 over inner, left, semi- and full joins. Further tests use `CHECK` constraints that `null`s pass, or that are `NOT VALID` or `NOT ENFORCED`, and in these nothing is removed. The last tests use ordinary keys whose columns a restriction fixes.

Differential testing: We generated 5,000 random schemas. Each has 3 to 6 columns over small domains, and 1 to 4 unique indexes, most of them partial. The predicates are random combinations of =, <>, `IN`, `NOT IN`, `IS [NOT] NULL`, boolean columns, `AND` and `OR`, and some indexes use `NULLS NOT DISTINCT`. The data repeat keys freely outside the scopes, because that is where unsound rewrites go wrong.

TABLE IV
DIFFERENTIAL TESTING: QUERIES RUN WITH THE PROVER OFF AND ON

run	queries	plans changed	results
joins and grouping, 7 shapes	200,000	20,737	0
`DISTINCT` over a table or join	200,000	27,809	0
grouping through joins	200,000	30,011	0
all shapes with `CHECK`s	400,000	75,975	0
main runs	1,000,000	154,532	0
bug #17975 shape	200,000	6,713	0
further runs	320,000	–	0

TABLE V
MUTANTS, EACH REMOVING ONE SAFEGUARD: RESULTS CHANGED OF 20,000 QUERIES

mutant	results changed
index predicates ignored	2,706
predicate from any clause (bug #17975)	442
`DISTINCT` proved with U() instead of $U^{=}$()	551
`GROUP BY` proved with U() instead of $U^{=}$()	232
both outer-join guards dropped	3
single-member classes as equalities*	34
semi-join columns as fixed*	11
partly known outer side*	0 of 40,000†
`CHECK` as if `null`s failed it	120
unvalidated `CHECK`s trusted	327

*errors of our first version †caught by a feature test. The semi-join error was found on TPC-DS q35.

We ran each random query with the setting off and on, under randomly chosen join and aggregation methods, and compared the two results as multisets.

Table IV lists the runs. The first run uses seven shapes of queries. Three are left, inner and semi-joins, and one has a scope established by a clause of the same join. The other three use `GROUP BY` over a table, over an inner join, and over the nullable side of a left join. The runs through joins use two and three tables that all carry random indexes. Their `ON` clauses may read two outer tables, and a third of the queries add a semi-join. The two runs with `CHECK`s add up to two random `CHECK` constraints per table. These are enforced, `NOT ENFORCED`, or added as `NOT VALID` after rows that violate them. No run changed a result.

The test can detect errors. Ignoring the index predicates would have removed 14,845 left joins in the first run and 78,619 `DISTINCT`s in the second, of which 5,269 and 23,796 would have been wrong on the data. Table V shows the mutants, each of which removes one safeguard. Bug #17975 needs a predicate that only another join's clause implies, as in `o JOIN t ON o.k = t.k JOIN u ON t.c = u.c` over `UNIQUE (k) WHERE c IS NOT NULL`. None of the seven shapes generates such a query, but a run of that shape catches the bug. Dropping only one of the two guards against `null`s from outer joins changes nothing, because the other guard catches the case. The mutant that enters an outer join from a partly known outer side needs an outer row that is not determined and that decides a match. The feature tests supply such data. All main runs had the `Inner Unique` setting on, which only adds to what the default does. Of the further 320,000 queries, 160,000 ran under the default setting, in two runs that changed 14,351 and 13,862 plans. In all, no result changed on 1,520,000 queries.

B. Decisions

Setup: This experiment asks whether the prover finds every rewrite that a catalog implies and no other, and what the deployed systems find instead. We took nine tables from four real datasets: Hockey, VOC, NCVoter with 1.02M records, and UniProt. On each table we declared up to 40 conditional keys, mined with the key-first miner of Section VI. We declared them as an application would, using filtered unique indexes only and no `CHECK`s. Keys that hold with `null` equal to `null` became indexes with `NULLS NOT DISTINCT`. Certain keys [35], which hold whatever value a `null` stands for, imply this property. We declared them with one index per pattern of `null`s on the key (Proposition 3). This gives 32 to 166 indexes per table. Nineteen UniProt indexes exceed PostgreSQL's limit of 8 kB per index row, and we left them out everywhere.

We generated 1,454 left joins and 1,454 `DISTINCT` queries whose predicates differ from the index predicates. Some refine a scope with an atom of another index, some extend a key by a column, some use the union of two scopes, and some are random controls. We asked the real optimisers, read their plans, and checked every decision on the data. Table VI shows the result.

What the systems find: Stock PostgreSQL 18 uses no partial unique index. PostgreSQL 15.3 is the last release before the fix for bug #17975, and it removes no join either, because it sets `predOK` only after join removal. It does mark 1,033 joins as `Inner Unique`, using one index at a time. SQLite 3.54 removes 33 joins by matching predicates word for word. On 14 queries it fails with an internal planner error. Two partial unique indexes with disjoint predicates on one column, and an `OR` of both predicates in the `ON` clause, trigger it.

What the prover finds: The patched planner removes 1,094 joins and agrees with implication from the whole catalog on all 1,454 queries. That is 61 joins more than one index at a time allows. Neither stock system removes `DISTINCT` over a base table. The patched planner removes 171, none of them wrongly. These are exactly the removals that implication from the whole catalog allows, and 54 more than one index at a time allows. The caption of Table VI also counts the rewrites that happen to be correct on this data. For each of the remaining ones, the catalog admits a table on which it is wrong, so no sound planner may apply it. A planner that ignores the `null` semantics uses each index for `DISTINCT` as if it had `NULLS NOT DISTINCT`. It allows 1,033 removals, and 167 of them are wrong. Of these, 113 are on NCVoter. There, keys with `null`s distinct over sparse name suffixes put all records without a suffix into one `DISTINCT` group.

When combining indexes matters: Combining indexes pays off on catalogs this dense. On the real workloads of

TABLE VI
REWRITES ALLOWED (WRONG) ON CATALOGS OF FILTERED UNIQUE INDEXES ONLY: 1,454 LEFT JOINS (1,208 REMOVABLE ON THE DATA) AND 1,454 DISTINCTS (985 REMOVABLE).

decider	left join	DISTINCT
PostgreSQL 18.6	0	0
PostgreSQL 15.3 (pre-#17975)[a]	0	0
SQLite 3.54	33 (14 errors)	0
one index at a time (model)	1,033	117
patched PostgreSQL 18.6	1,094	171
whole catalog (model)	1,094	171
one at a time, nulls ignored (model)	1,033	1,033 (167)

[a] marks 1,033 joins `Inner Unique`, none wrong.

Section VII-D, it never mattered. We built a version of the prover that tries one premise index at a time, while still combining restrictions, `NOT NULL` and `CHECK`s. With the `Inner Unique` setting on, it makes all 46 plan changes there that need a filtered key: 37 on Stack Exchange, 3 in Discourse, and 6 in application code.

CockroachDB: CockroachDB has no `NULLS NOT DISTINCT`. There, every key becomes indexes with `null`s distinct, one for each pattern of `null`s except the one where all key columns are `null` (Proposition 3). This catalog implies fewer `DISTINCT` removals. It implies more left-join removals, because the 19 oversized UniProt indexes fit in CockroachDB. On the same queries, the stock optimiser removes 6 left joins, all of them joins that can never match, and no `DISTINCT`. The port removes 1,146 of the 1,148 left joins and all 83 `DISTINCT`s that the catalog implies, none of them wrongly. One index at a time allows 1,087 and none. The port refuses the 2 that it misses, either at the search limit or because a candidate key is missing.

On open data: From the USA and Singapore archives of LakeBench [13], we took 285 tables whose first rows show a status-like column, an identifier, and columns that are partly `null`. Of these, 217 tables with 738 to 210,552 rows have verified conditional keys, 6,917 in all. We declared them as 15,173 indexes, of which 119 exceed the limit on index rows and were left out. We generated 29,408 left joins and as many `DISTINCT`s, as above. The patched planner agrees with implication from the whole catalog on every query. It removes 23,692 joins and 4,962 `DISTINCT`s, none of them wrongly. These are 1,483 and 963 more than one index at a time allows. Stock PostgreSQL removes none. SQLite removes 4,192 joins, and fails on 1,123 queries over 50 tables with the internal error described above. A planner that ignores the `null` semantics removes 22,171 `DISTINCT`s, and 3,121 of these removals, on 133 of the 217 tables, are wrong. On 40 of the tables, the CockroachDB port makes 4,535 of the 4,543 join removals and 225 of the 229 `DISTINCT` removals that the catalog implies. None of them is wrong, and it refuses the other 12.

C. Cost

We measured planning time on the 97 TPC-DS queries that PostgreSQL can plan, at scale factor 1, with the five business keys of current versions declared as partial unique indexes. Planning took 0.14 ms in the median. The prover added 0.026 ms in the median, about a fifth, and 0.29 ms at most, as medians of 15 runs. Most of this time goes to the pass through joins, which runs for every query that groups or removes duplicates.

As a stress test, we also declared the keys mined on the dimension tables that hold at scale factors 1 and 10 (Section VII-E). A unique index can state 3,996 of these 4,002 keys, and we declared all of them. With the five keys above, this gives 4,001 indexes, and up to 1,771 on one table. Planning then took 1.5 ms in the median without the prover. The prover added 0.15 ms in the median, about a tenth, and 2.8 ms at most. In the 95th percentile, planning with the prover took 2.4 times as long as without it. Without the premise cache of Section V, the maximum was 45 ms, on queries that join `item` with its 948 indexes and plan in 2 to 8 ms without the prover. Without the premise filter, the maximum was 830 ms. Two TPC-DS plans changed, q46 and q68. Their `GROUP BY` loses `ca_city` of a customer address that is joined on its primary key. No filtered key changed a plan, for the reason given in Section VII-D.

D. Gains

Table VII groups the workloads by who wrote their queries. For each workload, the table gives four numbers. The first is the queries that PostgreSQL can plan. The second counts the queries that name a table with a filtered unique index together with the columns of its predicate. For SEDE we give a stricter count, of the queries in which a table's own restriction implies the predicate. The third is the number of plans that change with the prover on. In parentheses are the changes that occur only if the filtered indexes are declared. The last column gives the speed-up of the changed queries that take more than 1 ms. We report medians of ten rounds that alternate the setting, with the data cached for SEDE. For soft deletion we report medians of five runs after two warm-up runs, and for the views one run after a warm-up run. All timings are under the default setting. The `Inner Unique` setting changes 24 more plans on Stack Exchange and one more in Discourse, all through filtered keys, and two over MusicBrainz. Every changed query returned the same result, compared in one transaction and without a trailing `LIMIT`, since a shorter grouping key may break ties differently.

Application code: Application code rarely filters on the scope of a filtered key. In seven applications we found 325 places in the code that read a table with a filtered key, including the SQL that ActiveRecord generates. Of these, 221 are point queries, upserts and uniqueness validations, so the indexes serve integrity first. On generated data, every changed query runs in less than 1 ms, except one Chatwoot query ($0.93\times$). The MusicBrainz server picks a primary alias in its own code, and ListenBrainz reaches artists by their identifiers. Discourse's administrators join e-mail addresses on `primary` in 22 reports. The prover proves the join unique in every one of them, yet only 2 plans change because of it, or 3 with the

TABLE VII
WORKLOADS BY WHO WROTE THE QUERIES

workload	written by	plan	scope	changed	speed-up
code of 7 applications	developers	90	–	6 (6)	0.93
2 dbt projects (left joins)	analysts	665	12	–	–
Discourse reports	administrators	722	39	29 (2)	0.96–1.06
MusicBrainz, ListenBrainz	developers	20	0	0	–
MetaBrainz forum	users	15	0	0	–
TPC-DS	benchmark	97	0	2 (0)	–
STATS-CEB	benchmark	146	40	0	–
Data Explorer queries	users	6,578	555	118 (13)	0.37–3.30
through a filtered key				13	0.91–1.07
SkyServer SQL form	astronomers	17,738	757	26 (0)	–
Wikimedia Quarry	editors	8,736	14	178 (0)	–
soft delete, tenants	us	5	5	5 (5)	1.01–5.75
TPC-DS through views, SF 10	us	73	73	73 (73)	0.91–1.43
MusicBrainz consumers	us	10	10	7 (7)	1.15–4.94

`Inner Unique` setting. The reports remove duplicates over common table expressions, expressions or aggregates that no key of a base table reaches. Of the 29 plan changes there, 27 come from primary keys carried through joins. dbt materialises its models, so the planner sees their joins only once. All 12 joins to scopes of current versions also fetch columns. TPC-DS declares keys of current versions but never selects current versions. In 40 of its 146 queries, STATS-CEB [8] selects a recommended scope, but it counts over joins on columns that are not keys. The workloads also exposed two gaps in translation, which we have closed: `smallint` enum columns compared with `integer` constants (GitLab), and indexes with `text_pattern_ops` (Mastodon). Users of their own MusicBrainz copies behave no differently: the 15 plannable queries posted on the MetaBrainz forum name no scope, and none of their plans changes.

Queries written against the data: Users who write queries against the data filter on the scopes more often. The SEDE corpus [11] holds 12,023 queries that users of the Stack Exchange Data Explorer wrote against the schema of Section VII-E. Of these, 6,578 translate from T-SQL and can be planned on the `stats.stackexchange.com` data with the 56 filtered unique indexes that our recommender proposed (Section VII-E). A third of them mention a table with such an index together with the columns of its predicate. In 555, the table's own restriction implies the predicate. Most often these are users with a location, accepted answers, close events and duplicate links. Under the default setting, the prover changes 118 plans. Of these, 105 use primary keys alone, carried through joins or removing `DISTINCT`, and 13 use a recommended key.

We timed the changed queries that take more than 1 ms. With 4 GB of shared buffers, which hold the 2.8 GB of data, 15 of these 104 queries run more than $1.1\times$ faster, up to $3.30\times$, all through primary keys. One runs more than $1.1\times$ slower, taking 13 ms instead of 5. The 10 timed changes through a filtered key run at 0.91 to $1.07\times$. With PostgreSQL's default of 128 MB of shared buffers, 10 queries run faster and 4 slower.

The `Inner Unique` setting changes 24 more plans, 142 in all, and 37 of them through a recommended key. Of these 37, 28 use a restriction to the key's scope directly. One uses a restriction that the planner derives from equalities with a constant (`PostHistoryTypeId = Pt.Id = 2`). The other 8 use joins on the accepted answer. That key excludes only `null` keys, so under $\mathrm{U}()$ it is an ordinary key, which PostgreSQL ignores because the index is partial. With the data cached, 8 of the 32 timed changes through a filtered key are then faster, up to $1.67\times$, and 3 are slower, down to $0.88\times$. At 128 MB, 13 of all changed queries run faster and 15 slower. The slower ones result from physical choices that the cost model makes with the new knowledge. It picks a hash join instead of a merge join over an index. Or, where a `GROUP BY` disappears and rows no longer need to arrive in key order, it picks a sequential scan instead of an index scan. These choices are costly when the data is not cached, and mostly cheap when it is. On Stack Exchange, the recommended keys pay off only through join costing. Whether that is worth the risk depends on the cache, so we leave `enable_conditional_unique_costing` to the administrator.

Astronomers who query the Sloan Digital Sky Survey through SkyServer [10] filter on scopes all the time, because its schema is built on them. `PhotoPrimary` is `PhotoObjAll` with `mode = 1`, and its documentation says that it holds one row per object on the sky, that is, `UNIQUE (thingID) WHERE mode = 1`. `SpecObj` is `SpecObjAll` with `sciencePrimary = 1`. We collected the distinct queries typed into its SQL form from January 2025 to July 2026, and 17,738 of them can be planned on its schema. Of these, 10,444 read science-primary spectra, but only 1,158 remove duplicates, group or left-join. The prover changes 26 plans, all through primary keys carried through joins. No query needs the documented key. In 376 queries, users left-join `SpecObj` on `bestObjID` as if it were a key of the science-primary spectra. In the data it is not, as 69 objects have two such spectra. A planner must prove uniqueness from declared constraints, not from what a schema is meant to say.

Users of Wikimedia's Quarry [9] behave in the same way. Of their queries, 8,736 can be planned on MediaWiki's schema, and 178 plans change. Most of these are a `GROUP BY` or `DISTINCT` on `page_title` within one namespace. There, the key `(page_namespace, page_title)` becomes a key on the title under the query's restriction, which stock PostgreSQL does not see. We mined 26 keys on Simple English Wikipedia that also hold on Afrikaans Wikipedia. They change no plan, and only 14 queries select their scopes.

Queries we wrote: Over MusicBrainz, we wrote queries of the kind a consumer of its data would write. They gain most where a partial index states a key outright ($4.94\times$, for distinct names and comments), and where an unused join to the primary alias is removed ($3.58\times$). One of these needs the `CHECK` constraint that gives every primary alias a locale. We also built soft-deleted and tenant-scoped data, with 1M users and 5M orders. There, removing a left join gains 1.64 and $4.46\times$, and removing a `DISTINCT` gains $5.75\times$. An earlier

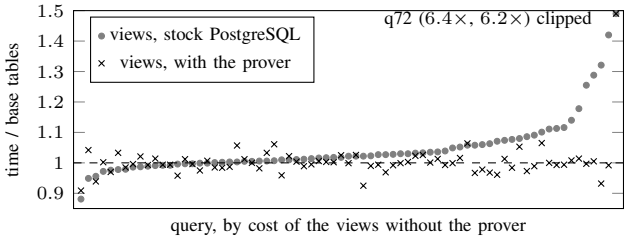


Fig. 2. TPC-DS at scale factor 10 through views that restate each history-keeping dimension with its current attributes: run time of the 73 queries that read one, relative to the same query on the base tables.

run with the same plans gave 3.65, 5.61 and 3.87×. These factors thus vary by up to 2× between runs, and the times include fetching up to 5M rows, which no rewrite changes. Over MusicBrainz, the `Inner Unique` setting changes two more plans, which become 1.2 to 1.8× faster. It changes none of the others.

Views over versioned data: Reporting layers often show historical facts together with current attributes. To test this pattern, we offered each history-keeping dimension of TPC-DS as a view of the same name. For `item`, the view is `SELECT h.*, c.i_brand AS cur_i_brand, ... FROM item h LEFT JOIN item c ON c.i_item_id = h.i_item_id AND c.i_rec_end_date IS NULL`. We ran the unchanged queries through these views, with the five keys of current versions declared, and compared them with the queries on the base tables (Fig. 2). At scale factor 10, the 73 queries that read such a dimension contain 141 joins added by the views. Stock PostgreSQL keeps all of them, and the prover removes all of them, with identical results. Query q72 is an exception: the view's `FROM` list prevents it from being flattened, whether or not the join is removed. It is fast again with `from_collapse_limit` set to 20. Without q72, the views cost 7% with stock PostgreSQL (836 against 783 s), and nothing with the prover (778 s). At scale factor 1, they cost 2%, or 17% with q6, whose time the views raise from 29 to 48 s. The joins are removed whatever their size, but what they cost depends on the dimension. TPC-DS dimensions are small next to its facts: at scale factor 10, `item` has 102,000 rows and `store_sales` 28.8 million.

E. Recommendation

Would a recommender find them?: We hid the declared indexes and generated candidate conditions from two sources. The schema proposes `IS [NOT] NULL` on nullable columns, boolean values, and the constants of columns that look like a status, type or kind. The workload contributes the atoms that its queries apply, with their constants. Of the 168 declared conditions on plain columns, half (84) only exclude `null` keys and follow from the key itself. Of the other 84, the schema recovers 78, at the level of column and kind of atom, since the constants come from the data. The workload recovers 68 with their constants, and both sources together recover 79. The misses are conditions that require a non-empty string (`<> ''`), and two predicates that our parser does not read. On the tables with these indexes, the schema proposes 2,776 candidate atoms and the workload 524. The workload is thus the sharper source, and the schema the broader one. Choosing keys under these conditions needs data, which these applications do not publish. We leave this to deployments with real instances.

Stability under updates: We mined the 15 dimension tables of TPC-DS at scale factor 1. We found 11,988 keys of at most three columns under conditions of at most two atoms, 10,645 of them conditional. We kept the 4,002 that also hold at scale factor 10. DuckDB's generator produces no refresh sets, so we emulated the dimension maintenance of the specification on scale factor 1, in three rounds. In each round, 5% of the current versions of the history-keeping dimensions get a successor version, 5% of the rows of the other maintained dimensions are updated in place, and 2.5% new entities are added. New attribute values are drawn from the existing values of each column. After three rounds, 79% of the 3,126 conditional keys on maintained tables that held at both scale factors still hold. Of those that held at scale factor 1 only, 58% still hold. Mined keys without conditions fare worse (45% and 26%), as they are mostly accidents of small tables. The keys on the seven static dimensions all hold. Holding at a second scale factor is therefore a useful but weak filter. Of the keys it keeps, 79% survive, against 64% of all conditional keys, but it keeps only 36% of the keys that survive. The emulation samples columns independently, so it breaks correlations that the real generator keeps. These rates are therefore lower bounds.

A public dataset: We used the Stack Exchange data of `stats.stackexchange.com` from April 2024, with 5.6M rows in 8 tables. The schema proposed 457 conditions: values of type, status and flag columns, `null` tests, and conjunctions of one of each. HPIValid found the keys of at most three columns on each scope, under both semantics. This gave 107 recommendations in 147 s. Under $\mathrm{U}()$, a column that is `null` throughout a scope is trivially a key. Requiring 50 rows where the key is not `null` removed 46 such candidates. We also mined the data before 2020, and 97 of the 103 recommendations that carry dates still hold in 2024.

By our reading, 81 recommendations state invariants of the platform. A revision has one history row per kind of change. A post has one initial body, title and tag set, and at most one vote that accepts an answer. An answer is accepted by at most one question, which is a $\mathrm{U}^{=}()$ key among values that are not `null`. Two posts are linked once per type of link. A user earns a tag badge once per class. The other 26 recommendations follow correlations, such as licence versions that follow time. The hold-out data rejects only 2 of them, together with 4 invariants that turned out to be false: a post can become community wiki, or a hot network question, twice.

All Stack Exchange sites share the same schema. We therefore checked the 101 recommendations that hold in 2024 on three other sites (dba, academia and travel, with 0.5M posts). Of the 99 that can be tested, 86 hold on all three.

The 13 that fail include 7 of the 24 correlations, and 6 that we had read as invariants. A post can be protected or tweeted twice, and on some sites it can receive several spam votes. Other sites are thus a sharper filter than later data. But the 17 correlations that pass follow the platform's history, which all sites share. Choosing among recommendations remains a decision for the owner of the schema.

Keys from data: By Proposition 9, an existing algorithm for unique column combinations finds the keys under a fixed condition. We took seven real tables with 21 conditions each, under both semantics, which gives 288 scopes that are not trivial. HyUCC and HPIValid of Desbordante [12] returned the same minimal keys as our miner on every scope where our miner finished within 20 s. HPIValid took 0.1 to 61 ms per table in the median (Appendix D).

VIII. Related Work

Uniqueness in optimisation: Semantic query optimisation rewrites queries using integrity constraints [37], [16]. Filtered unique indexes are such constraints, and the question is which rewrites they allow. PostgreSQL first answered this question unsoundly, and then not at all (bug #17975). Rowley proposed the distinction between base restrictions and join clauses that we use for uniqueness, and it is under discussion for estimates [15]. Paulley and Larson [44] derive the uniqueness of query results from keys. Simmen et al. [47] and Neumann and Moerkotte [40] propagate functional dependencies for order optimisation, and Kossmann et al. [38] survey optimisation based on dependencies. Godfrey et al. [32] exploit characterisations of the data that behave like constraints.None of these works treats keys that hold on part of a table. Deriving the keys of a join from those of its inputs goes back to Darwen [21] and Paulley [22]. Section V adds what filtered keys need: a premise for each relation that holds under its own restrictions, and a semantics ($\mathrm{U}()$ or $\mathrm{U}^{=}()$) for each step. The recommender of Section VI fits with lazy discovery of dependencies for the optimiser, driven by the workload [23].

Partial indexes: Stonebraker proposed partial indexes [50], and Seshadri and Swami generalised them [49]. Later work concerns how to select them and how to use them as access paths. Choosing a partial index for a query restriction is the implication test of predicate subsumption [39], [31]. Using a partial *unique* index to prove uniqueness is an implication between constraints, which is what this paper decides.

Declared but unenforced keys: DB2 eliminates joins using declared constraints [17]. Snowflake, Databricks and Oracle remove joins using primary and unique keys marked `RELY` [18], [19], [20], and Databricks also removes `DISTINCT`. The system trusts such keys but does not check them. These keys hold on the whole table and may be false. Filtered unique indexes, in contrast, are enforced but hold on part of a table. The question is therefore not whether to trust them, but what they imply.

Testing optimisers: Our differential testing follows the oracles for logic bugs of Rigger and Su [24], [25], [26]. We compare the planner with the rewrites off and on, and use mutants to show that the test detects errors.

Keys with nulls and conditions: Possible and certain keys [35], [36] and key sets [51], [33], [34] give keys a semantics for `null`. Here $\mathrm{U}(K)$ is the possible key. A certain key implies $\mathrm{U}^{=}(K)$, but filtered unique indexes cannot express it. Embedded uniqueness constraints [54] and contextual keys [53] restrict keys by completeness or by context. CFDs [5], [28], and extended CFDs with disjunction and inequality [6], bind functional dependencies to patterns over complete relations. Over complete relations that are sets, a conditional key with a constant condition is a CFD. What our model adds is `null` in conditions and keys, which is what filtered indexes mostly state. The hardness of Theorems 6 and 7 comes from the same effect of finite domains that makes CFD implication coNP-complete, whereas it is polynomial over infinite domains [28]. Here, `null` against a value, and an `IN`-list, act as a domain of two or three values. Discovery of unique column combinations [43], [3], [1] and of CFDs [30] informs our miner.

IX. Conclusion

A planner can trust a filtered unique index when it proves uniqueness from the table's own restrictions, all its unique indexes and its `CHECK` constraints, under the `null` semantics of the rewrite. For the predicates of almost all real decisions, join-shaped rewrites can then be decided in polynomial time. Under the same conditions, group-shaped rewrites are hard only through nullable columns that the query leaves open.

A prover built on these results fits into PostgreSQL's planner with 38 changed lines, and into CockroachDB's with 4. It adds tens of microseconds to planning in the median. It allowed exactly what real catalogs imply, and it changed no result on 1,000,000 random queries. What it gains depends on who writes the queries. Application code and reports use filtered keys for integrity, and gain little. Users' queries over Stack Exchange gain up to $3.3\times$, mostly through keys carried through joins. Views over versioned data lose every join they add.

Some limits remain. Range predicates and expression keys are not translated, and derived tables are not reasoned about. Self-join elimination is not yet connected to the prover, because it needs the clauses that established uniqueness.

Appendix A
Proofs for Reasoning

Proof of Theorem 7, cases on K. Both cases reduce from CNF satisfiability with `NOT NULL` columns $A_1, \ldots, A_n, B$ and the goal $\{\tau\} : \mathrm{U}(A_1 \cdots A_n B)$, so a violating pair shares a value v_i on each A_i ("x_i is true" iff $v_i = 1$). For $\notin$: τ has $\square$ everywhere, and clause constraints $\{\rho\} : \mathrm{U}(B)$ have $\rho(A_i) = \notin\{1\}$ for a positive and $\notin\{0\}$ for a negative literal; the entry fails iff the literal is true, and a value other than 0, 1 makes no literal true. For $\in$: $\tau(A_i) = \in\{0, 1\}$, and $\rho(A_i) = \in\{0\}$ for a positive and $\in\{1\}$ for a negative literal. $\square$

Proposition 10. *Deciding* $\Sigma \models \{\langle\rangle\} : \mathrm{U}(B)$*, and* $\Sigma \models \{\langle\rangle\} : \mathrm{U}^{=}(B)$*, is coNP-complete already when* Σ *consists of a single constraint whose condition is built from* `IS NULL`*,* `IS NOT NULL`*,* `AND` *and* `OR`*.*

Proof. Membership in coNP follows from Lemma 2. Hardness is by reduction from CNF satisfiability with clauses $c_1, \ldots, c_m$ over $x_1, \ldots, x_n$: take nullable columns $A_1, \ldots, A_n$ and $B \in NN$, read "A_i is not `null`" as "x_i is true", and let $\Sigma = \{C : \mathrm{U}(B)\}$, where C has one template per clause c_j, with $\perp$ on A_i if x_i occurs positively in c_j and $\top$ if negatively: a record matches it iff its assignment falsifies c_j. A pair violating the goal agrees on B and satisfies Σ iff one of its records lies outside the scope of C, that is, satisfies the CNF. Hence $\Sigma \models \varphi$ iff the CNF is unsatisfiable. As $B \in NN$, the same holds for $\mathrm{U}^{=}(B)$. □

Separations: Let $\sigma = \{\langle d\colon \perp\rangle\} : \mathrm{U}(k)$ over columns k, d, and suppose a set Σ of embedded uniqueness constraints $E : U$ (records total on E are unique on $U \subseteq E$) were equivalent to it. The table A with two records $(k\colon 1, d\colon \texttt{null})$ violates σ, so it violates some $E : U$ in Σ, with $d \notin E$ as both records are `null` on d. The table B with $(k\colon 1, d\colon \texttt{null})$ and $(k\colon 1, d\colon 1)$ satisfies σ but violates $E : U$, as $E \subseteq \{k\}$. For $\sigma = \{\langle s\colon \in\{c\}\rangle\} : \mathrm{U}(k)$ take A with two records $(k\colon 1, s\colon c)$ and B with two records $(k\colon 1, s\colon c')$: both are total and duplicate, so every embedded constraint that A violates B violates too, while B satisfies σ. For CFDs, two identical records satisfy every CFD and violate every conditional `UNIQUE` whose scope contains them; a single record with $A = a$ and $B \neq b$ violates the CFD $(A = a \to B = b)$ and satisfies every conditional `UNIQUE`.

Appendix B
Proof of the Determination Lemma

Fix a database that satisfies the constraints, and two rows u, v of the join of the block that agree on G, `null` equal to `null`. A row of the join consists of one part per relation outside the inner sides of semi- and anti-joins: a row of the relation, at a position of its own, or `null`s if an outer join null-extended it. We show by induction on the order in which relations become determined that u and v have the same part of each determined relation S.

Entered through G. S is unique with `null` equal to `null` on its columns K in G among its rows that pass its restrictions and `CHECK` constraints, and no outer join can null these columns. So the parts of S in u and v are rows of S, not `null`s; both pass the restrictions, which are applied below every join; they agree on K with `null` equal to `null`, because u and v agree on G; hence they are the same row.

Entered through an equivalence class. A column s of S is in an equivalence class with two or more members, one of them a column x of a determined relation R or a column of G that no outer join nulls. The class's equalities are strict and applied wherever both columns are available, so in every row that contains a row of S, its s is the value of x and not `null`. By the induction hypothesis (or because u and v agree on G), x is the same in u and v. If both contain a row of S, the two rows pass S's restrictions and agree on all columns by which S was entered, with values, so $\mathrm{U}()$ uniqueness of S on them makes them one row. It remains that both contain a row of S or neither does. If no outer join can null S, both do. Otherwise S lies on the nullable side of left joins, and the rule requires each of their outer sides to be determined: whether the nullable side of such a join is matched or null-extended is a function of the outer row, which is the same in u and v, and of the database, which is fixed.

Entered through an outer join clause. The clause equates a column s of S, on the nullable side, with a column x of a determined relation on the outer side, and the whole outer side is determined. Given the outer row, which u and v share, the rows of S that the clause admits pass S's restrictions and have s equal to the value of x; by $\mathrm{U}()$ uniqueness at most one does. So both u and v contain it, or both are null-extended, as whether a match exists depends on the outer row and the fixed database only.

Inner sides of semi- and anti-joins contribute no part and no row: a row of the join is determined by its parts outside them. If every other relation is determined, u and v have the same part of every relation, hence are the same combination of rows at the same positions, that is, the same row of the join; so no two rows agree on G. For `GROUP BY` with aggregates, a relation S determined by the remaining grouping columns G' has the same part in all rows of a group of G', so its grouping columns do not split groups; G' is kept non-empty, since grouping by nothing yields a row even for an empty input. □

Appendix C
The Axiomatisation

Fix the set Γ of constants of an instance. The *profile* of a record has, per column, `null`, a constant of Γ, or $*$ for a value outside Γ; it is the scope of exactly one template τ_p, with $\in\{c\}$, $\perp$ and $\notin\Gamma$ as entries. For a goal with key part κ (that is, $\mathrm{U}(K)$ or $\mathrm{U}^{=}(K)$) and two profiles p, q, let $c_\kappa(p, q)$ be the *canonical pair*: records of profiles p and q that agree on K and carry different fresh values wherever both have $*$ outside K; the *cell* (p, q) is vacuous if no pair of these profiles agrees on K. Writing $\top$ for $\notin\emptyset$, D for a condition, $\tau[A{\mapsto}e]$ for τ with entry e on A, and dropping templates with $\perp$ on a `NOT NULL` column:

(T) $\Rightarrow \emptyset : \kappa$

(S) $(D \cup \{\tau[A{\mapsto}\top], \tau[A{\mapsto}\perp]\}) : \kappa \Rightarrow (D \cup \{\tau[A{\mapsto}\square]\}) : \kappa$

(V) $(D \cup \{\tau[A{\mapsto}\notin S\cup\{c\}], \tau[A{\mapsto}\in\{c\}]\}) : \kappa \Rightarrow (D \cup \{\tau[A{\mapsto}\notin S]\}) : \kappa$

(I) $(D \cup \{\tau[A{\mapsto}\in\{c\}] : c \in S\}) : \kappa \Rightarrow (D \cup \{\tau[A{\mapsto}\in S]\}) : \kappa$

(P) $(D \cup \{\tau\}) : \kappa,\ (D \cup \{\tau'\}) : \kappa,\ \{\tau, \tau'\} : \kappa \Rightarrow (D \cup \{\tau, \tau'\}) : \kappa$

(C1) $\sigma \Rightarrow \{\tau_p\} : \kappa$ if $c_\kappa(p, p)$ violates σ

(C2) $\{\tau_p\} : \kappa,\ \{\tau_q\} : \kappa,\ \sigma \Rightarrow \{\tau_p, \tau_q\} : \kappa$ if $c_\kappa(p, q)$ violates σ

In (S) the second template is omitted on a `NOT NULL` column, in (V) $c \notin S$, and in (C1), (C2) σ is any constraint with constants in Γ, omitted when the cell is vacuous.

Theorem 11 (Axiomatisation)**.** *The rules (T), (S), (V), (I), (P), (C1) and (C2) form a sound and complete* 3*-ary inference system for the implication of conditional* `UNIQUE` *constraints with conditions built from* `=, <>, IN, NOT IN, IS [NOT] NULL, AND` *and* `OR`*, under both* `null` *semantics and any mix of them.*

The proofs below use the following notation. Values come from an infinite domain $\mathbb{V}$. We write $\top$ for $\notin\emptyset$, drop templates with $\bot$ on a column of NN (their scope is empty), and write $\mathrm{sc}(C)$ for the records matching a condition C and $\mathrm{consts}(\cdot)$ for the constants occurring in entries. For a key part κ with key K, records t, t' *agree under* κ, $t \approx_\kappa t'$, if for all $A \in K$: $t[A] = t'[A] \neq \mathtt{null}$ (for $\mathrm{U}(K)$), respectively $t[A] = t'[A]$ with `null` equal to `null` (for $\mathrm{U}^{=}(K)$). A pair (t, t') *violates* $C : \kappa$ if $t, t' \in \mathrm{sc}(C)$ and $t \approx_\kappa t'$; this is symmetric in t, t'.

A. Profiles and cells

Fix a finite $\Gamma \subseteq \mathbb{V}$. A Γ*-profile* p assigns to each column an element of $\Gamma \cup \{*, \bot\}$, and not $\bot$ to a column of NN. The profile $\mathrm{prof}(t)$ of a record has $t[A]$ where $t[A] \in \Gamma$, $\bot$ where $t[A] = \mathtt{null}$, and $*$ otherwise; the template τ_p has $\in\{c\}$ where p has c, $\bot$ where p has $\bot$, and $\notin\Gamma$ where p has $*$. A profile p *matches* τ if every record of profile p does, and $\mathrm{Prof}(C)$ is the set of profiles matching a template of C.

Lemma 12 (Profiles)**.** *(a)* $\mathrm{sc}(\tau_p) = \{t : \mathrm{prof}(t) = p\}$*. (b) If* $\mathrm{consts}(\tau) \subseteq \Gamma$*, whether* t *matches* τ *depends only on* $\mathrm{prof}(t)$*; so* $\mathrm{sc}(C) = \biguplus_{p \in \mathrm{Prof}(C)} \mathrm{sc}(\tau_p)$ *when* $\mathrm{consts}(C) \subseteq \Gamma$.

Proof. (a) holds by definition. (b) On a column A, $t[A]$ matches $\square$ always, $\bot$ iff $\mathrm{prof}(t)(A) = \bot$, $\in S$ with $S \subseteq \Gamma$ iff $\mathrm{prof}(t)(A) \in S$, and $\notin S$ with $S \subseteq \Gamma$ iff $\mathrm{prof}(t)(A) \notin S \cup \{\bot\}$, as a value outside Γ lies outside S. Profile scopes are disjoint by (a). □

The cell (p, q) is *vacuous for* κ if some $A \in K$ has $p(A) \neq q(A)$, or $p(A) = q(A) = \bot$ and $\kappa = \mathrm{U}(K)$. Otherwise the *canonical pair* $c_\kappa(p, q) = (t_1, t_2)$ has, on $A \in K$, $t_1[A] = t_2[A]$ equal to $p(A)$ if $p(A) \in \Gamma$, to `null` if $p(A) = \bot$, and to a value $v_A \notin \Gamma$ if $p(A) = *$; on $A \notin K$, the constant or `null` that each profile prescribes, and where it has $*$ a value $u^i_A \notin \Gamma$; all of v_A, u^1_A, u^2_A are distinct. Then $\mathrm{prof}(t_1) = p$, $\mathrm{prof}(t_2) = q$ and $t_1 \approx_\kappa t_2$, and a cell is vacuous iff no records of profiles p, q agree under κ.

Lemma 13 (Weakest pair)**.** *Let* (p, q) *be non-vacuous for* κ *and* $\mathrm{consts}(\sigma) \subseteq \Gamma$*. If* $c_\kappa(p, q)$ *violates* σ*, so does every pair* (t, t') *with* $\mathrm{prof}(t) = p$*,* $\mathrm{prof}(t') = q$ *and* $t \approx_\kappa t'$.

Proof. Let $\sigma = D : \lambda$ with key L and $c_\kappa(p, q) = (t_1, t_2)$. By Lemma 12(b), $t, t' \in \mathrm{sc}(D)$ as t_1, t_2 are. Let $A \in L$, where t_1, t_2 agree under λ. If $A \in K$: for $p(A) = q(A) = \bot$, κ is $\mathrm{U}^{=}(K)$, $t_1[A], t_2[A]$ are `null`, so λ is $\mathrm{U}^{=}(L)$ and $t[A] = t'[A] = \mathtt{null}$ agree; for a constant, $t[A] = t'[A]$ is that constant; for $*$, $t[A], t'[A]$ are values made equal by $t \approx_\kappa t'$. If $A \notin K$: were $p(A) = *$ or $q(A) = *$, $t_1[A], t_2[A]$ would be different and not both `null`, contradicting their agreement; so $t[A] = t_1[A]$ and $t'[A] = t_2[A]$ are prescribed by the profiles, and agree. □

Lemma 14 (Cells)**.** *Let* $\varphi = C_\varphi : \kappa$ *and* $\Gamma \supseteq \mathrm{consts}(\Sigma \cup \{\varphi\})$*. Then* $\Sigma \models \varphi$ *iff for all* $p, q \in \mathrm{Prof}(C_\varphi)$ *with* (p, q) *non-vacuous, some* $\sigma \in \Sigma$ *is violated by* $c_\kappa(p, q)$.

Proof. ($\Leftarrow$) If r satisfies Σ and (t, t') in r violates φ, then $p = \mathrm{prof}(t), q = \mathrm{prof}(t') \in \mathrm{Prof}(C_\varphi)$, (p, q) is non-vacuous, and the σ violated by $c_\kappa(p, q)$ is violated by (t, t') (Lemma 13). ($\Rightarrow$) If no $\sigma \in \Sigma$ is violated by $c_\kappa(p, q) = (t_1, t_2)$ for a non-vacuous cell, the table $\{t_1, t_2\}$ respects NN, satisfies Σ (its pairs are (t_1, t_2) and (t_2, t_1)) and violates φ. □

B. Soundness and completeness

Proof of soundness. (T): the empty condition has empty scope. (S), (V), (I): premise and conclusion have the same key part and scope, since on A the entries of the premise partition the records matching the entry of the conclusion ($\top$ and $\bot$ for $\square$, only $\top$ on a `NOT NULL` column; $\notin S \cup \{c\}$ and $\in\{c\}$ for $\notin S$; the $\in\{c\}$ for $\in S$). (P): let (t, t') violate the conclusion and $X = \mathrm{sc}(D)$. If both lie in $X \cup \mathrm{sc}(\tau)$ or both in $X \cup \mathrm{sc}(\tau')$, a premise is violated; otherwise one lies in $\mathrm{sc}(\tau') \setminus X$ and the other in $\mathrm{sc}(\tau) \setminus X$, violating $\{\tau, \tau'\} : \kappa$. (C1): a violating pair of $\{\tau_p\} : \kappa$ has both profiles p (Lemma 12(a)) and agrees, so (p, p) is non-vacuous and the pair violates σ (Lemma 13). (C2): a violating pair with equal profiles violates a premise $\{\tau_p\} : \kappa$ or $\{\tau_q\} : \kappa$; otherwise, up to symmetry, its profiles are p, q and it violates σ. □

For a template τ with $\mathrm{consts}(\tau) \subseteq \Gamma$ let $T(\tau)$ be the set of τ_p with p matching τ, and $\mu(\tau) = \sum_A \mu(\tau(A))$ with $\mu(\square) = |\Gamma| + 2$, $\mu(\notin S) = |\Gamma \setminus S|$, $\mu(\in S) = |S| - 1$, $\mu(\bot) = 0$; $\mu(\tau) = 0$ iff τ is a profile template.

Lemma 15 (Merging)**.** *From* $(D \cup T(\tau)) : \kappa$*, rules (S), (V), (I) derive* $(D \cup \{\tau\}) : \kappa$.

Proof. Induction on $\mu(\tau)$; for $\mu(\tau) = 0$, $T(\tau) = \{\tau\}$. Otherwise choose A with $\mu(\tau(A)) > 0$ and the premise templates $\tau_1, \ldots, \tau_k$ of the rule whose conclusion template is τ: $\tau[A{\mapsto}\top]$ and, off NN, $\tau[A{\mapsto}\bot]$ if $\tau(A) = \square$; $\tau[A{\mapsto}\notin S \cup \{c\}]$, $\tau[A{\mapsto}\in\{c\}]$ for some $c \in \Gamma \setminus S$ if $\tau(A) = \notin S$; $\tau[A{\mapsto}\in\{c\}]$, $c \in S$, if $\tau(A) = \in S$ with $|S| \geq 2$. Then $\mu(\tau_i) < \mu(\tau)$ and $T(\tau) = T(\tau_1) \uplus \cdots \uplus T(\tau_k)$. With $E_i = D \cup \{\tau_1, \ldots, \tau_i\} \cup T(\tau_{i+1}) \cup \cdots \cup T(\tau_k)$, the induction hypothesis derives $E_i : \kappa$ from $E_{i-1} : \kappa$, and the rule turns $E_k : \kappa$ into $(D \cup \{\tau\}) : \kappa$. □

Proof of completeness. Let $\Sigma \models \varphi = C_\varphi : \kappa$, $\Gamma = \mathrm{consts}(\Sigma \cup \{\varphi\})$; if $C_\varphi = \emptyset$ use (T). Let $P = \mathrm{Prof}(C_\varphi)$ and $T_P = \{\tau_p : p \in P\} = \bigcup_{\theta \in C_\varphi} T(\theta)$. (1) For $p \in P$, (C1) derives $\{\tau_p\} : \kappa$, with the premise that Lemma 14 provides if (p, p) is non-vacuous. (2) For $p \neq q$ in P, (C2) derives $\{\tau_p, \tau_q\} : \kappa$ likewise. (3) For $Q \subseteq T_P$ with $|Q| \geq 3$, by induction on $|Q|$, (P) with distinct $\tau, \tau' \in Q$ and $D = Q \setminus \{\tau, \tau'\}$ derives

$Q : \kappa$; so $T_P : \kappa$ is derived. (4) With $C_\varphi = \{\theta_1, \ldots, \theta_m\}$ and $F_j = \{\theta_1, \ldots, \theta_j\} \cup T(\theta_{j+1}) \cup \cdots \cup T(\theta_m)$, Lemma 15 derives $F_j : \kappa$ from $F_{j-1} : \kappa$; $F_0 = T_P$ and $F_m = C_\varphi$. □

Every derived constraint has the goal's key part and members of Σ occur only as the σ of (C1) and (C2), so the rules are complete for each semantics alone. Derivations can be doubly exponentially long ($|P|$ is exponential and step (3) uses all subsets), consistent with coNP-hardness; the decision procedures of Section IV do not search for derivations.

C. A derivation and a counterexample

Example 16 (A derived key)**.** A table has the filtered unique indexes

```
UNIQUE (s) WHERE s = 'a', UNIQUE (s) WHERE s
    <> 'a', UNIQUE ((1)) WHERE s IS NULL,
```

that is $\sigma_1 = \{\langle s{:}\in\{a\}\rangle\} : \mathrm{U}(s)$, $\sigma_2 = \{\langle s{:}\notin\{a\}\rangle\} : \mathrm{U}(s)$ and $\sigma_3 = \{\langle s{:}\bot\rangle\} : \mathrm{U}(\emptyset)$ (at most one row with s `null`). Together they enforce `UNIQUE NULLS NOT DISTINCT (s)`, $\varphi = \{\langle\rangle\} : \mathrm{U}^{=}(s)$, so `SELECT DISTINCT s` can drop its `DISTINCT`. Here $\Gamma = \{a\}$ and the profiles are a, $*$ and $\bot$, with templates $\langle s{:}\in\{a\}\rangle$, $\langle s{:}\notin\{a\}\rangle$ and $\langle s{:}\bot\rangle$; the cells of two different profiles are vacuous, as the profiles differ on the key s. Writing conditions by their entries on s:

1	$\{\in\{a\}\} : \mathrm{U}^{=}(s)$	(C1), σ_1; $c(a,a)$: $s = a, a$
2	$\{\notin\{a\}\} : \mathrm{U}^{=}(s)$	(C1), σ_2; $c(*,*)$: $s = v, v$
3	$\{\bot\} : \mathrm{U}^{=}(s)$	(C1), σ_3; $c(\bot,\bot)$: $s = \mathtt{null}, \mathtt{null}$
4	$\{\in\{a\}, \notin\{a\}\} : \mathrm{U}^{=}(s)$	(C2), 1, 2; vacuous
5	$\{\in\{a\}, \bot\} : \mathrm{U}^{=}(s)$	(C2), 1, 3; vacuous
6	$\{\notin\{a\}, \bot\} : \mathrm{U}^{=}(s)$	(C2), 2, 3; vacuous
7	$\{\in\{a\}, \notin\{a\}, \bot\} : \mathrm{U}^{=}(s)$	(P), 5, 6, 4; $D = \{\bot\}$
8	$\{\top, \bot\} : \mathrm{U}^{=}(s)$	(V), 7; $S = \emptyset$, $c = a$
9	$\{\Box\} : \mathrm{U}^{=}(s)$, i.e. φ	(S), 8

Step 3 uses that two `null`s agree under $\mathrm{U}^{=}()$; the cell $(\bot, \bot)$ would be vacuous for a $\mathrm{U}()$ goal. Each σ_i is needed: without it, its cell has no premise.

Example 17 (A counterexample)**.** Take the two scopes of Example 4: $\sigma_1 = \{\langle d{:}\bot\rangle\} : \mathrm{U}^{=}(k)$, $\sigma_2 = \{\langle d{:}\top\rangle\} : \mathrm{U}(k)$, and the goal $\varphi = \{\langle x{:}1\rangle\} : \mathrm{U}(k)$, so $\Gamma = \{1\}$. Following the proof of completeness, the cells within the scope of φ are checked one by one. The cell of $p = (k{:}*, d{:}\bot, x{:}1)$ and $q = (k{:}*, d{:}*, x{:}1)$ is non-vacuous (both have a value on k), and its canonical pair is

$$t_1 = (k{:}v,\ d{:}\mathtt{null},\ x{:}1), \qquad t_2 = (k{:}v,\ d{:}u,\ x{:}1).$$

It violates neither premise: t_2 is outside the scope of σ_1 and t_1 outside that of σ_2. So (C2) has no premise for this cell, the derivation cannot be completed, and by Lemma 14 the table $\{t_1, t_2\}$ is a counterexample: it satisfies both indexes and has two rows with $x = 1$ and the same k. This is the evidence a planner obtains when it refuses to drop a join probing k among the rows with $x = 1$.

APPENDIX D
DISCOVERY

For a table r, let $\mathrm{Th}(r)$ be the set of conditional `UNIQUE` constraints that r satisfies; a *cover* of $\mathrm{Th}(r)$ is a finite subset that implies all of it.

Theorem 18 (Finite covers)**.** *Let Γ be the constants occurring in r. The constraints $\{\tau_p, \tau_q\} : \kappa$ that hold on r, for Γ-profiles p, q and key parts $\kappa \in \{\mathrm{U}(K), \mathrm{U}^{=}(K)\}$ with K minimal, form a finite cover of $\mathrm{Th}(r)$; it implies also the constraints with constants that do not occur in r.*

The cover is finite because `NOT IN` names "a value that r does not contain" with one entry. But it shows that the unrestricted problem is degenerate: every value of r lies in Γ, so the profile of a row records all its values, and conditions can single out any set of rows. The cover above enumerates pairs of rows—for tables of at most five rows and three columns it has 146 members in the median and up to 4,232—and the largest scope on which a key K is unique is found trivially by keeping one row per value of K. Discovery is meaningful only over a bounded condition language. Let $\mathcal{L}_a$ be the templates with at most a entries other than $\Box$, each an *atom* `A = c`, `A <> c` (c in the active domain of A), `A IS NULL` or `A IS NOT NULL`, and let $\mathrm{Disc}_a(r)$ list, for every $\tau \in \mathcal{L}_a$ and both semantics, the minimal keys K such that $\tau : \mathrm{U}(K)$, respectively $\tau : \mathrm{U}^{=}(K)$, holds on r. As a constraint implies the same constraint with a larger key, $\mathrm{Disc}_a(r)$ covers the constraints with conditions in $\mathcal{L}_a$ that hold on r.

Theorem 19 (Complexity of discovery)**.** *Let r be a table over R, and d the largest active domain of a column.*

(a) *Whether a given constraint holds on r is decidable in polynomial time.*
(b) *$|\mathcal{L}_a| \le \big(|R|\,(2d+3)\big)^a$, polynomial for fixed a.*
(c) *Given τ and k, deciding whether $\tau : \mathrm{U}(K)$ holds on r for some K with $|K| \le k$ is NP-complete and, parameterised by k, W[2]-complete, already for $\tau = \langle\rangle$; the same holds for $\mathrm{U}^{=}()$.*
(d) *For given τ and semantics, the minimal keys are the minimal transversals of a hypergraph on R with at most $|r|^2$ edges, computable in polynomial time; for $\tau = \langle\rangle$ over tables without `null`, enumerating them is equivalent under parsimonious reductions to enumerating the minimal transversals of a hypergraph [4].*

Hence, for fixed a, $\mathrm{Disc}_a(r)$ is computable in incremental quasi-polynomial time [29], in output-polynomial time if minimal transversals can be enumerated in output-polynomial time, and for $a = 0$ only then [27]. With keys bounded by a fixed size, it takes polynomial time.

Part (c) follows from the unconditional case [4], [48], part (d) from the observation that K is a key within a scope iff it meets the set of columns on which every two rows of the scope fail to agree (proofs below). Our miner works with $a = 2$ and keys of at most three columns, so its search is polynomial;

Algorithm 1 FIXC(r, C, κ, k, H_0): minimal keys of at most k columns

$S \leftarrow$ the rows of r matching C; **if** $|S| < 2$ **return** $\{\emptyset\}$
replace each `null` in S by a value of its own ($\mathrm{U}()$) or by one value $\perp$ ($\mathrm{U}^{=}()$)
$\pi_A \leftarrow$ clusters of ≥ 2 rows of S equal on A, for each $A \in R$
$H \leftarrow \{D \in H_0 : \text{its two rows lie in } S\} \cup \{D(t,t') : t, t' \text{ consecutive in a cluster}\}$
$T \leftarrow \{\emptyset\}$; **for each** $D \in \min H$: $T \leftarrow \mathrm{Add}_k(T, D)$; $V \leftarrow \emptyset$
while some $X \in T \setminus V$ **do**
  if the refinement of π by X has a cluster with rows t, t' **then**
    if $D(t,t') = \emptyset$ **return** $\emptyset$ ▷ no key exists
    $H \leftarrow H \cup \{D(t,t')\}$; $T \leftarrow \mathrm{Add}_k(T, D(t,t'))$
  else $V \leftarrow V \cup \{X\}$
  end if
end while
return T (and H, for reuse)

Algorithm 2 DISCA(r, a, k): all templates of $\mathcal{L}_a$

for $\kappa \in \{\mathrm{U}(), \mathrm{U}^{=}()\}$ and $\tau \in \mathcal{L}_a$ by increasing number of atoms **do**
  $\tau' \leftarrow \tau$ without its last atom ▷ a wider scope
  $(K_\tau, H_\tau) \leftarrow$ FIXC$(r, \{\tau\}, \kappa, k, H_{\tau'})$ ▷ $H_{\langle\rangle}$ from $H_0 = \emptyset$
end for
return all $\tau : \kappa(K)$ with $K \in K_\tau$

what makes discovery hard is unbounded key size, as for unconditional keys, not the conditions.

Fixing the condition: By part (d) and Proposition 9, discovery for a fixed condition is discovery of unique column combinations on the encoded rows in scope. Algorithm 1 follows the hybrid scheme of [43]: disagreement sets of neighbouring rows in the position list indexes (PLIs) give candidate keys as minimal transversals; each candidate is validated by intersecting PLIs, and a failing candidate yields a pair whose disagreement set updates the candidates by one step of Berge's algorithm [2], $\mathrm{Add}_k(T, D) = \min\big(\{X \in T : X \cap D \neq \emptyset\} \cup \{X \cup \{A\} : X \in T,\, X \cap D = \emptyset,\, A \in D,\, |X| < k\}\big)$.

Proposition 20. FIXC *returns exactly the minimal keys of at most k columns, and* DISCA *returns* $\mathrm{Disc}_a(r)$ *restricted to such keys.*

Proof. Every disagreement set in H belongs to two rows of the scope, so every key meets it. A returned X is a validated key and a minimal transversal of H, so no proper subset of X is a key. A minimal key K with $|K| \leq k$ is a transversal of H and contains a minimal one, X, with $|X| \leq k$, which Add_k keeps; X is validated, so it is a key and $X = K$. Each failed validation adds a set that the candidate misses, so H grows and the loop ends. Inherited sets in DISCA are kept only if both rows lie in the narrower scope. □

The cost is dominated by validations, $O(|S|\,|X|)$ each; on random tables both procedures agree with exhaustive search (Section D-B).

Fixing the key: Our second miner goes the other way. For a candidate key K (at most three columns) the pairs of records that agree on K are computed once by grouping; a condition makes $C : \mathrm{U}(K)$ hold iff no such pair has both records in its scope, so all valid templates of at most two atoms—a frequent constant of a low-cardinality column, a column present, a column missing—are found in one pass over the atom sets the pairs share, and single-constant templates are merged into `IN`-lists whose cross pairs are clean. Results implied by the declared indexes or by more general results are dropped by the prover of Section V, so the recommendation is irredundant.

The difficulty is not search but accidental keys. On TPC-DS's six-row `call_center` table, 342 unconditional keys hold; checking them on a second scale factor keeps 156, and on small dimension tables almost all. Ranking by scope size, identifier-like columns and simplicity of the condition helps; a recommender should, in addition, prefer keys that the workload can use.

A. Proofs

Proof of Theorem 18. Every listed constraint holds on r by definition, and there are finitely many, as there are finitely many Γ-profiles and keys. Let $\varphi = C : \kappa$ hold on r, with key K_φ, and suppose the cover Σ does not imply it. With $\Gamma' = \Gamma \cup \mathrm{consts}(\varphi)$, Lemma 14 yields Γ'-profiles $p', q' \in \mathrm{Prof}(C)$ whose cell is non-vacuous and whose canonical pair (t_1, t_2) satisfies Σ. Let p, q be the Γ-profiles of t_1, t_2 and $Z \supseteq K_\varphi$ the set of columns on which t_1, t_2 agree under κ; write κ_Z for the key part of the same semantics with key Z. The pair (t_1, t_2) violates $\{\tau_p, \tau_q\} : \kappa_Z$. If that constraint held on r, so would one with a minimal key $K \subseteq Z$, which belongs to Σ and is also violated by (t_1, t_2)—a contradiction. So r has rows t, t' at different positions with Γ-profiles in $\{p, q\}$ that agree on Z under κ. Rows of r carry only values of Γ, so these profiles contain no $*$; hence the corresponding record of the canonical pair carries only values of Γ or `null`, and its Γ'-profile, p' or q', equals its Γ-profile. Thus t, t' have Γ'-profiles in $\{p', q'\} \subseteq \mathrm{Prof}(C)$, lie in $\mathrm{sc}(C)$ by Lemma 12, and agree on $K_\varphi \subseteq Z$ under κ: (t, t') violates φ on r, a contradiction. □

The construction was checked mechanically: on 400 random tables and 2,000 random constraints, with constants inside and outside the tables, "φ holds on r" coincided with "the cover implies φ".

Proof of Theorem 19. (a) Collect the rows of the scope, drop for $\mathrm{U}(K)$ those with `null` on K, and look for two rows with equal projections on K (with `null` as a value for $\mathrm{U}^{=}(K)$) by sorting or hashing. (b) A template of $\mathcal{L}_a$ chooses at most a columns and, for each, one of at most $2d+2$ atoms; counting the choice "no atom" gives the bound. (c) Membership in NP: guess K and check it by (a). For hardness take $\tau = \langle\rangle$ and tables without `null`, on which $\mathrm{U}(K)$ and $\mathrm{U}^{=}(K)$ both say that K is a unique column combination. Deciding whether a unique column combination of size at most k exists is NP-complete and, parameterised by k, W[2]-complete [4]; finding a minimum one is NP-hard [48]. (d) For rows t, t' let $D_\kappa(t, t')$ be the set of columns on which they do not agree under κ. Then $\tau : \kappa$ holds on r iff $K \cap D_\kappa(t, t') \neq \emptyset$

for all rows t, t' at different positions in the scope of τ, i.e. iff K is a transversal of the hypergraph of these sets; the minimal keys are its minimal transversals. If some set is empty, no key exists. For $\tau = \langle\rangle$ and tables without `null` these are the minimal unique column combinations, whose enumeration is equivalent to transversal enumeration under parsimonious reductions [4]. For the consequences: $\mathrm{Disc}_a(r)$ consists of $2|\mathcal{L}_a|$ transversal problems, each built in time $O(|r|^2|R|)$; the duality test of Fredman and Khachiyan runs in quasi-polynomial time and, when it fails, returns a point from which a new minimal transversal is obtained, so minimal transversals are enumerated in incremental quasi-polynomial time [29]; an output-polynomial algorithm for transversals gives one for each problem and hence for their union, whose output is the sum of theirs; for $a = 0$ the converse holds as minimal unique column combinations can be enumerated in output-polynomial time only if transversals can [27], [4]. With $|K| \leq k$ fixed, all $O(|R|^k)$ candidate keys are checked by (a) for each template. □

Proof of Proposition 9. After the replacement, two records agree on K in the plain sense iff they agree under the chosen semantics, so $C : \kappa(K)$ holds on r iff K is unique on the records matching C. □

B. Experiments

Correctness: FIXC agreed with exhaustive search over all subsets of columns on $5{,}000$ random tables of up to five columns and twelve rows with `null`s and duplicates, random conditions with $=$, <>, `IN`, `NOT IN` and `IS [NOT] NULL`, both semantics and key bounds $k \in \{1, 2, \infty\}$, and on $1{,}500$ wider tables (four to eight columns, up to 40 rows) with up to 44 minimal keys each; DISCA with inherited disagreement sets agreed with FIXC from scratch on all $42{,}958$ templates of $\mathcal{L}_2$ over 500 random tables.

Cost: On seven real tables (Table VIII) we ran FIXC for the empty condition and 20 single-atom conditions (the most frequent constants, `IS NULL` and `IS NOT NULL` of the sparsest columns), under both semantics, with keys of at most three columns, against a level-wise baseline that validates every candidate whose subsets are not keys (single-threaded Python, same PLIs, same output on all 294 runs). FIXC needs 6–85 times fewer validations and is faster on 238 of the 294 runs, with median speed-ups of 1.6–5.7 on six tables and up to 88 on UniProt. It loses on Hockey.Teams (median 0.35): with about 300 minimal keys per condition, maintaining the transversals costs more than the validations it saves, which are cheap on 1,519 rows. Inheriting disagreement sets in DISCA (70–86 single-atom templates per table) saved at most 13% of the time against FIXC from scratch, and nothing on most tables: the seeds from PLI neighbours already find most disagreement sets, and validation dominates. Inheritance is therefore a minor optimisation.

Against UCC algorithms: On the same seven tables and 21 conditions under both semantics (288 scopes with at least two records; UniProt sampled to 20,000 of its rows), we wrote each encoded scope to a file and ran HyUCC and HPIValid of Desbordante 2.4.1 (C++, one thread) [12]. The two returned the same keys on all 288 scopes; FIXC without a size bound returned them on the 236 it finished within 20 s, and with at most three columns on the 281 it finished. HPIValid took 0.1–61 ms per table in the median, HyUCC 0.4–$50{,}091$ ms: under $\mathrm{U}^{=}()$ on UniProt, where the scopes have about 76,000 minimal keys in the median, HyUCC needed up to 21 minutes. FIXC with keys of at most three columns took 6.4 times as long as HPIValid in the median, loading included. Widening UniProt from 10 to 100 columns raised HPIValid from 0.5 to 45 ms. Our contribution to discovery is the reduction, not a new algorithm for unique column combinations.

TABLE VIII
FIXC AGAINST LEVEL-WISE VALIDATION, KEYS OF ≤ 3 COLUMNS: MEDIANS OVER 42 RUNS (21 CONDITIONS, TWO SEMANTICS); H. = HOCKEY, V. = VOC.

table	rows	cols	keys	FixC ms (val.)	levels ms (val.)
H.Master	7,761	31	263	462 (359)	788 (2,268)
H.Coaches	1,812	14	4	14 (5)	55 (427)
H.Teams	1,519	27	310	704 (356)	204 (3,133)
H.Goalies	4,278	23	46	101 (53)	420 (2,005)
V.voyages	8,131	23	37	97 (37)	558 (1,094)
V.seafarers	4,468	10	3	10 (3)	15 (73)
UniProt (20k)	20,000	100	396	1,655 (410)	11,587 (24,744)

Scaling the rows of the 30-column UniProt exports from 16,000 to 512,000 (empty condition and three `IS [NOT] NULL` conditions, both semantics; 40 scopes) kept all answers equal. HPIValid grew from 2.5 to 54 ms and HyUCC from 18 to $1{,}479$ ms on the empty condition under $\mathrm{U}()$, while reading the file grew from 0.4 to 16 s and dominated both. FIXC with keys of at most three columns needed 0.7 s at 16,000 rows and 16 s at 256,000, and exceeded 20 s at 512,000 on the large scopes.

On 40 of the LakeBench tables of Section VII-B, cut to the miner's key columns and up to 12 condition columns (1,630 scopes of up to 163,414 rows), all four agreed on every scope and FIXC always finished; in the median per table HPIValid took 0.4 ms, HyUCC 4.3 ms and FIXC with at most three columns 77 ms.

APPENDIX E
FURTHER EXPERIMENTS

Scaling of the decision procedure: On 30-column schemas with random conditional keys of one or two atoms, our Python implementation of the canonical-pair search needs 0.15, 0.37, 2.2 and 15 ms in the median for $|\Sigma| = 10$, 100, $1{,}000$ and $10{,}000$ at $b = 4$, against 0.34–244 ms for a SAT encoding; for $|\Sigma| = 100$ it grows from 0.19 ms at $b = 0$ to 669 ms at $b = 16$, where SAT stays at about 2 ms. The polynomial case of Theorem 5 is the $b = 0$ column.

A rewrite layer outside the planner: A catalog-reading rewrite layer in Python reproduced all 2,588 decisions on the eight tables whose indexes PostgreSQL can build (on UniProt some index rows exceed 8 kB), and the 80 sampled

rewrites it emitted returned the original results. Reading a table's constraints took 1–11 ms and deciding and rewriting a query 3.2 ms in the median in Python. The rewrites ran $1.8\times$ (`DISTINCT`) and $3.5\times$ (left join) faster in the median, and removing the join 10.7–$32.3\times$ on NCVoter. These runs used catalogs with all key classes of the miner, not filtered unique indexes only.

Mining the TPC-DS workload: The queries are also a source of conditions. The local predicates of each table occurrence (conjuncts of `WHERE` and inner-join `ON` clauses on that table alone, kept as SQL including ranges) give 162 distinct conditions on 22 tables in 96 of the 99 queries, extracted in 1.5 s. Checked by grouping at scale factors 1 and 10, 124 of the 181 key demands of the queries—the columns a block joins, groups or semi-joins on—hold under their condition, but 118 contain the primary key, and only 3 hold because of the condition, all on scopes of two to four rows that the query parameters select. Running FIXC under every condition on a dimension table (keys of at most two columns, 14 s in all) finds 1,983 keys that hold only under their condition, 1,382 of them also at scale factor 10, and 844 of these on scopes of fewer than 100 rows. Of the 794 under conditions of equalities and `IN`-lists, 232 hold for every value of the condition columns—d_{dom} within each month, for instance—and so are composite keys that include those columns (here $(d_{year}, d_{moy}, d_{dom})$) rather than conditional ones; the others depend on the parameter values. Mining the workload finds scopes cheaply, but a recommender needs a significance filter, and the selective, parameterised predicates of a benchmark are not where filtered unique indexes live.

ACKNOWLEDGMENT

Claude (Anthropic), through Claude Code, was used for this work. It generated text for the article, code for the PostgreSQL prover and CockroachDB port, experiment scripts, random schema and query generators (Sections V and VII); it ran the experiments, compiled their results, drew the tables and figures, checked the reductions of Section IV by brute force on random instances, and reviewed drafts. The authors directed the work and are responsible for the correctness and originality of the paper.

REFERENCES

[1] Z. Abedjan, L. Golab, F. Naumann. Profiling relational data: a survey. *VLDB J.* 24(4), 2015.

[2] C. Berge. *Hypergraphs: Combinatorics of Finite Sets*. North-Holland, 1989.

[3] J. Birnick, T. Bläsius, T. Friedrich, F. Naumann, T. Papenbrock, M. Schirneck. Hitting set enumeration with partial information for unique column combination discovery. *Proc. VLDB Endow.* 13(11):2270–2283, 2020.

[4] T. Bläsius, T. Friedrich, M. Schirneck. The complexity of dependency detection and discovery in relational databases. *Theor. Comput. Sci.* 900:79–96, 2022.

[5] P. Bohannon, W. Fan, F. Geerts, X. Jia, A. Kementsietsidis. Conditional functional dependencies for data cleaning. In *ICDE*, pp. 746–755, 2007.

[6] L. Bravo, W. Fan, F. Geerts, S. Ma. Increasing the expressivity of conditional functional dependencies without extra complexity. In *ICDE*, pp. 516–525, 2008.

[7] T. Döhmen, R. Geacu, M. Hulsebos, S. Schelter. SchemaPile: a large collection of relational database schemas. *Proc. ACM Manag. Data* 2(3), 172:1–172:25, 2024.

[8] Y. Han, Z. Wu, et al. Cardinality estimation in DBMS: a comprehensive benchmark evaluation. *PVLDB*, 15(4):752–765, 2021.

[9] H. Triedman. wikidb: queries from Wikimedia's Quarry. Hugging Face dataset `htriedman/wikidb`, 2023.

[10] A. S. Szalay, J. Gray, et al. The SDSS SkyServer: public access to the Sloan Digital Sky Server data. In *SIGMOD*, 2002.

[11] M. Hazoom, V. Malik, B. Bogin. Text-to-SQL in the wild: a naturally-occurring dataset based on Stack Exchange data. In *NLP4Prog Workshop at ACL*, 2021.

[12] G. Chernishev et al. Desbordante: from benchmarking suite to high-performance science-intensive data profiler. In *CODS-COMAD*, pp. 234–243, ACM, 2025.

[13] Y. Deng et al. LakeBench: a benchmark for discovering joinable and unionable tables in data lakes. *PVLDB*, 17(8):1925–1938, 2024.

[14] K. R. Carlsen. Postgres wont remove useless joins, when the UNIQUE index is partial. pgsql-general mailing list, January 2019.

[15] C. Yan. Restrict partial unique index estimates to base quals. pgsql-hackers mailing list, patch v1, July 2026.

[16] U. S. Chakravarthy, J. Grant, J. Minker. Logic-based approach to semantic query optimization. *ACM Trans. Database Syst.* 15(2):162–207, 1990.

[17] Q. Cheng, J. Gryz, F. Koo, T. Y. C. Leung, L. Liu, X. Qian, K. B. Schiefer. Implementation of two semantic query optimization techniques in DB2 Universal Database. In *VLDB*, pp. 687–698, 1999.

[18] Snowflake. *Join elimination*. https://docs.snowflake.com/en/user-guide/join-elimination.

[19] Databricks. *Query optimization using primary key constraints*. https://docs.databricks.com/aws/en/sql/user/queries/query-optimization-constraints.

[20] Oracle. *Database SQL Tuning Guide 19c*, chapter Query Transformations.

[21] H. Darwen. The role of functional dependence in query decomposition. In C. J. Date, H. Darwen, *Relational Database Writings 1989–1991*, Addison-Wesley, 1992.

[22] G. N. Paulley. *Exploiting Functional Dependence in Query Optimization*. PhD thesis, University of Waterloo, 2000.

[23] J. Kossmann, D. Lindner, F. Naumann, T. Papenbrock. Workload-driven, lazy discovery of data dependencies for query optimization. In *CIDR*, 2022.

[24] M. Rigger, Z. Su. Testing database engines via pivoted query synthesis. In *OSDI*, pp. 667–682, 2020.

[25] M. Rigger, Z. Su. Detecting optimization bugs in database engines via non-optimizing reference engine construction. In *ESEC/FSE*, pp. 1140–1152, 2020.

[26] M. Rigger, Z. Su. Finding bugs in database systems via query partitioning. *Proc. ACM Program. Lang.* 4(OOPSLA), 211:1–211:30, 2020.

[27] T. Eiter, G. Gottlob. Identifying the minimal transversals of a hypergraph and related problems. *SIAM J. Comput.* 24(6):1278–1304, 1995.

[28] W. Fan, F. Geerts, X. Jia, A. Kementsietsidis. Conditional functional dependencies for capturing data inconsistencies. *ACM Trans. Database Syst.* 33(2):6, 2008.

[29] M. L. Fredman, L. Khachiyan. On the complexity of dualization of monotone disjunctive normal forms. *J. Algorithms* 21(3):618–628, 1996.

[30] W. Fan, F. Geerts, J. Li, M. Xiong. Discovering conditional functional dependencies. *IEEE Trans. Knowl. Data Eng.* 23(5):683–698, 2011.

[31] J. Goldstein, P.-Å. Larson. Optimizing queries using materialized views: a practical, scalable solution. In *SIGMOD*, 2001.

[32] P. Godfrey, J. Gryz, C. Zuzarte. Exploiting constraint-like data characterizations in query optimization. In *SIGMOD*, 2001.

[33] S. Hartmann, U. Leck, S. Link. On Codd families of keys over incomplete relations. *Comput. J.* 54(7):1166–1180, 2011.

[34] M. Hannula, X. Li, S. Link. Controlling entity integrity with key sets. *J. Comput. Syst. Sci.* 136:195–219, 2023.

[35] H. Köhler, S. Link, X. Zhou. Possible and certain SQL keys. *Proc. VLDB Endow.* 8(11):1118–1129, 2015.

[36] H. Köhler, U. Leck, S. Link, X. Zhou. Possible and certain keys for SQL. *VLDB J.* 25(4):571–596, 2016.

[37] J. J. King. QUIST: a system for semantic query optimization in relational databases. In *VLDB*, pp. 510–517, 1981.

[38] J. Kossmann, T. Papenbrock, F. Naumann. Data dependencies for query optimization: a survey. *VLDB J.* 31(1), 2022.

[39] P.-Å. Larson, H. Z. Yang. Computing queries from derived relations. In *VLDB*, 1985.
[40] T. Neumann, G. Moerkotte. An efficient framework for order optimization. In *ICDE*, pp. 461–472, 2004.
[41] PostgreSQL Global Development Group. *PostgreSQL Documentation*: CREATE INDEX; Partial indexes. https://www.postgresql.org/docs/, accessed 2026.
[42] D. Rowley. Don't use partial unique indexes for unique proofs in the planner. PostgreSQL commit 7fcd7ef2a9, bug #17975, June 2023.
[43] T. Papenbrock, F. Naumann. A hybrid approach for efficient unique column combination discovery. In *BTW*, 2017.
[44] G. N. Paulley, P.-Å. Larson. Exploiting uniqueness in query optimization. In *ICDE*, pp. 68–79, 1994.
[45] SQLite. *Partial indexes*. https://sqlite.org/partialindex.html, accessed 2026.
[46] Microsoft. *SQL Server documentation*: CREATE INDEX; Create filtered indexes. https://learn.microsoft.com/sql/, accessed 2026.
[47] D. Simmen, E. Shekita, T. Malkemus. Fundamental techniques for order optimization. In *SIGMOD*, 1996.
[48] A. Skowron, C. Rauszer. The discernibility matrices and functions in information systems. In R. Słowiński (ed.), *Intelligent Decision Support*, Theory and Decision Library D 11, pp. 331–362. Kluwer, 1992.
[49] P. Seshadri, A. Swami. Generalized partial indexes. In *ICDE*, pp. 420–427, 1995.
[50] M. Stonebraker. The case for partial indexes. *SIGMOD Rec.* 18(4):4–11, 1989.
[51] B. Thalheim. On semantic issues connected with keys in relational databases permitting null values. *J. Inf. Process. Cybern.* 25(1/2):11–20, 1989.
[52] P. White. Optimizer limitations with filtered indexes. https://www.sql.kiwi/2013/04/optimizer-limitations-with-filtered/, 2013.
[53] Z. Wei, S. Link, J. Liu. Contextual keys. In *ER*, pp. 266–279, 2017.
[54] Z. Wei, U. Leck, S. Link. Discovery and ranking of embedded uniqueness constraints. *Proc. VLDB Endow.* 12(13):2339–2352, 2019.